TWO-WHEELER ACCIDENT REPORTING AND TRACKING SYSTEM

A PROJECT REPORT

Submitted by

MOHAMMED SAAJID KHAN M A

(241401056)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND BUSINESS SYSTEMS



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "**Two-Wheeler Accident Reporting and Tracking System**" is the bonafide work of **MOHAMMED SAAJID KHAN M A (241401056)**, who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. K. R. Sowmia, Ph.D.,
HEAD OF THE DEPARTMENT
Associate Professor & Head
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

M. L. Alphin Ezhil Manuel,
COURSE FACULTY
Assistant Professor (SG)
Department of Computer Science and Business Systems
Rajalakshmi Engineering College
Thandalam – 602 105

Submitted for the End Semester Examination (Individual Project) held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, **Mohammed Saajid Khan M A (241401056)**, hereby declare that the project report titled "**Two-Wheeler Accident Reporting and Tracking System**", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report (see Section 5.9 for the AI assistance used).

Place: Chennai
Date:

SIGNATURE OF THE STUDENT
Mohammed Saajid Khan M A
241401056

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman **Mr. S. Meganathan, B.E., F.I.E.,** Chairperson **Dr. Thangam Meganathan, Ph.D.,** and Vice-Chairman **Mr. Abhay Shankar Meganathan, B.E., M.S.,** for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, **Dr. S. N. Murugesan, M.E., Ph.D.,** Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank **Dr. K. R. Sowmia, Ph.D.,** Associate Professor & Head, Department of Computer Science and Business Systems, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **M. L. Alphin Ezhil Manuel,** Assistant Professor (SG), for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

MOHAMMED SAAJID KHAN M A

ABSTRACT

Two-wheeler riders are exposed to road accidents, and in small organisations the details of an accident are usually passed on by phone, written in a notebook or kept on a personal phone. Records are scattered, nobody can see which cases are still open or how severe they are, severe accidents are not treated differently from minor ones, and the actions taken afterwards are not linked to the accident.

The Two-Wheeler Accident Reporting and Tracking System is a low-code application built on Zoho Creator to replace this practice. Low-code suits the problem because recording, tracking and summarising accidents is a data-entry and workflow task that forms, reports and dashboard components cover directly. The application has four linked forms, Rider, Vehicle, Accident Report and Action Taken, joined by lookup fields, with an auto-number report number and mandatory and no-duplicate validations. Reports include an Open Accident Cases list that hides closed cases and two pivot charts, by severity and by status, which are combined on an Accident Dashboard page built in the UI Builder. Two form workflows hold three Deluge scripts: one sets the status of Severe and Fatal reports to Under Review before saving, one e-mails the administrator with the severity in the subject, and one is meant to create a follow-up event in Zoho Calendar through a Creator connection. Access is controlled by a role hierarchy (CEO, Safety Officer, Rider), two permission sets and a data sharing rule.

The application was tested with three riders, three vehicles and seven accident reports. The notification e-mail was received and a severe report was saved as Under Review automatically. The follow-up event in Zoho Calendar is created at the accident date and time. The application was published to the Stage and Production environments (version 1.1, 31 October 2026). Access control was tried with two test users (a Rider and a Safety Officer) but not tested systematically for every role, and no timing measurements or user feedback were collected, so no quantitative benefit is claimed. These points are listed in the defect log and in the future enhancements.

Keywords: Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Accident Reporting

PROJECT AT A GLANCE

Attribute	Details
Project Title	Two-Wheeler Accident Reporting and Tracking System
Application Domain	Road safety / accident reporting for two-wheeler riders
Student Name & Register No.	MOHAMMED SAAJID KHAN M A – 241401056
Zoho Creator Edition	Trial edition
Application Name (Link Name)	two-wheeler-accident-reporting-and-tracking-system
Published / Live URL	https://mohammedsaajidkhanma202344.zohocreatorportal.in
Demo Credentials (per role)	Rider: mohammedsaajidkhan@gmail.com Safety Officer: zoho788@gmail.com
Number of Forms / Subforms	4 (Rider, Vehicle, Accident Report, Action Taken) / 0
Number of Reports (by type)	List: 5 · Kanban: 0 · Calendar: 0 · Pivot/Chart: 2 (Accidents by Severity, Accidents by Status)
Number of Pages / Dashboards	1 (Accident Dashboard)
Number of Workflows	Form: 2 · Schedule: 0 · Approval: 0 · Blueprint: 0 · Batch: 0
Deluge Functions / Custom Actions	3 scripts inside the form workflows (DS-01, DS-02, DS-03); no standalone function
Integrations	Zoho Calendar (follow-up event created through the Calendar API with invokeurl); e-mail through sendmail
Roles & Profiles Defined	Roles: CEO, Safety Officer, Rider · Permission sets: Safety Officer, Rider (plus built-in Read, Write)
Environments Used	Development → Stage (1.1, 31 Oct 2026) → Production (1.1, 31 Oct 2026)
Date of Production Deployment	31 October 2026

TABLE OF CONTENTS

PAGE

ABSTRACT ..... **v**

PROJECT AT A GLANCE ..... **vi**

LIST OF TABLES ..... **ix**

LIST OF FIGURES ..... **ix**

LIST OF ABBREVIATIONS ..... **xi**

INTRODUCTION **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope of the Project 2

1.5 Justification for a Low-Code Approach 2

1.6 Organisation of the Report 3

LITERATURE SURVEY AND SYSTEM STUDY **4**

2.1 Study of the Existing System 4

2.2 Limitations of the Existing System 4

2.3 Low-Code versus Traditional Development 5

2.4 Literature Survey 6

2.5 Emerging Trends in Low-Code Development 6

2.6 Proposed System 7

REQUIREMENT ANALYSIS **8**

3.1 Stakeholders and User Roles 8

3.2 Functional Requirements 8

3.3 Non-Functional Requirements 9

3.4 Hardware and Software Requirements 10

3.5 Use Case Model 10

SYSTEM DESIGN **13**

4.1 System Architecture 13

4.2 Data Model 13

4.3 Form Design 15

4.4 Process and Workflow Design 17

4.5 User Interface and Navigation Design 19

4.6 Access Control Design 20

IMPLEMENTATION **22**

5.1 Application Setup 22

5.2 Forms and Data Capture 23

5.3 Reports 26

5.4 Pages and Dashboards 28

5.5 Deluge Scripting 29

5.6 Workflows and Automation 31

5.7 Integrations 32
5.8 User Management and Role-Based Access 35
5.9 AI and Analytics Features 38
5.10 Pro-Code Extension 38
5.11 Mobile and Portal Access 38
TESTING AND DEBUGGING 39
6.1 Testing Strategy 39
6.2 Test Cases 39
6.3 Debugging Deluge Scripts 40
6.4 Defect Log 42
6.5 User Acceptance Testing 44
DEPLOYMENT, SECURITY AND GOVERNANCE 45
7.1 Environments and Deployment Strategy 45
7.2 Performance Optimisation 46
7.3 Security Features 47
7.4 Governance and Administration 47
RESULTS AND DISCUSSION 49
8.1 Application Output 49
8.2 Achievement of Objectives 50
8.3 Comparison with the Existing System 51
8.4 Limitations of the Low-Code Approach 51
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 52
CONCLUSION AND FUTURE ENHANCEMENTS 53
9.1 Conclusion 53
9.2 Future Enhancements 53
9.3 Learning Outcomes and Career Relevance 53
REFERENCES 54
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 55
APPENDIX B – COURSE OUTCOME MAPPING 57
APPENDIX C – COMPLETE DELUGE SOURCE CODE 58
C.1 Workflow “Notify on New Accident Report” (Accident Report form) 58
C.2 Workflow “Auto Review for Severe Accidents” (Accident Report form) 58
C.3 Workflow “Notify on New Accident Report” – calendar action 59
APPENDIX D – USER MANUAL 60
D.1 Accessing the Application 60
D.2 Instructions for the Administrator 60
D.3 Instructions for the Safety Officer 60
D.4 Instructions for the Rider 60

LIST OF TABLES

PAGE

Table 2.1: Comparison of Low-Code and Traditional Development for the Accident Reporting Application 5
Table 2.2: Summary of Literature Survey 6
Table 3.1: Stakeholders and User Roles 8
Table 3.2: Functional Requirements 8
Table 3.3: Non-Functional Requirements 9
Table 3.4: Hardware and Software Requirements 10
Table 3.5: Use Case Description – Submit Accident Report (UC-03) 11
Table 3.6: Use Case Description – Update Report Status (UC-04) 12
Table 3.7: Use Case Description – Record Action Taken (UC-05) 12
Table 4.1: Entities and Relationships 15
Table 4.2: Field Specification – Rider Form 15
Table 4.3: Field Specification – Vehicle Form 16
Table 4.4: Field Specification – Accident Report Form 16
Table 4.5: Field Specification – Action Taken Form 17
Table 4.6: Accident Report Status Values and Transitions 19
Table 4.7: Role–Permission Matrix 20
Table 5.1: Application Set-up Summary 22
Table 5.2: Forms Implemented 23
Table 5.3: Reports Implemented 26
Table 5.4: Components of the Accident Dashboard 28
Table 5.5: Deluge Scripts Implemented 30
Table 5.6: Workflows Implemented 31
Table 5.7: Integrations Implemented 33
Table 5.8: Data Sharing Rules 35
Table 6.1: Test Cases 39
Table 6.2: Summary of Test Results 40
Table 6.3: Debugging Observations 41
Table 6.4: Defect Log 42
Table 6.5: User Acceptance Feedback 44
Table 7.1: Deployment Checklist 45
Table 7.2: Performance Optimisation Results 46
Table 7.3: Security Controls Implemented 47
Table 8.1: Mapping of Objectives to Outcomes 50
Table 8.2: Existing System versus Proposed Application 51
Table 8.3: Observed Application State from the Screenshots 51
Table A.1: Feature Coverage Checklist 55
Table B.1: Mapping of Course Outcomes to Report Chapters 57

LIST OF FIGURES

PAGE

Figure 2.1: As-Is Process Flow of Informal Accident Reporting 4
Figure 3.1: Use Case Diagram of the Two-Wheeler Accident Reporting and Tracking System 11
Figure 4.1: System Architecture of the Two-Wheeler Accident Reporting and Tracking System 13

Figure 4.2: Entity–Relationship Diagram of the Application (* = mandatory) 14
Figure 4.3: Workflow / Process Flow for Submitting an Accident Report 18
Figure 4.4: Status Lifecycle of an Accident Report 19
Figure 4.5: Navigation Structure of the Application 20
Figure 5.1: New Application in the Creator Builder 23
Figure 5.2: Application Menu with the Accident Reports Section Expanded 23
Figure 5.3: Rider Form in the Form Builder 24
Figure 5.4: Vehicle Form in the Form Builder 24
Figure 5.5: Accident Report Form in the Form Builder 25
Figure 5.6: Action Taken Form in the Form Builder 25
Figure 5.7: Rider Form in Live Mode (sample data) 25
Figure 5.8: Accident Report Form in Live Mode 26
Figure 5.9: Filter Configuration of the Open Accident Cases Report 27
Figure 5.10: Open Accident Cases Report (4 of 4 records) 27
Figure 5.11: Detail View of a Record (report 1, status Closed) 27
Figure 5.12: Accidents by Severity Pivot Chart in the Report Builder 28
Figure 5.13: Accidents by Status Pivot Chart (live) 28
Figure 5.14: UI Builder Element Palette of the Accident Dashboard Page 29
Figure 5.15: Accident Dashboard – Severity and Status Charts 29
Figure 5.16: Accident Dashboard – Status Chart and Open Accident Cases Report 29
Figure 5.17: Deluge Editor with the Notification Script (DS-01) 31
Figure 5.18: Creating the Workflow “Notify on New Accident Report” 31
Figure 5.19: Choosing the “Validations on form submission” Event 32
Figure 5.20: Workflow “Notify on New Accident Report” (Enabled) 32
Figure 5.21: Workflow “Auto Review for Severe Accidents” (Enabled) 32
Figure 5.22: Connections List Showing Zoho Calendar Connection (Connected) 34
Figure 5.23: Connection Details and Access Turned On for the Application 34
Figure 5.24: Deluge “zoho integration” Block for Zoho Calendar → createEvent 34
Figure 5.25: Zoho Calendar Settings Showing the Calendar UID 35
Figure 5.26: Permission Sets: Read, Rider, Safety Officer and Write 36
Figure 5.27: Safety Officer Permission Set 36
Figure 5.28: Rider Permission Set 36
Figure 5.29: Role Hierarchy: CEO → Safety Officer → Rider 37
Figure 5.30: Data Sharing Rule “Share Rider Reports with Safety Officer” 37
Figure 5.31: All Accident Reports Seen by the Administrator (4 of 4 Records) 37
Figure 5.32: All Accident Reports Seen by the Safety Officer Before Any Rider Report (0 of 0) 37
Figure 6.1: First Version of Script DS-03 Saved as an Action of the Workflow (before the BUG-06 fix) 42
Figure 6.2: Form Validation Message on Year of Manufacture (BUG-01) 42
Figure 6.3: Open Accident Cases before the Filter Was Applied (5 of 5, BUG-03) 43
Figure 6.4: Dashboard Pie Chart Using Actual Data (BUG-04) 43
Figure 6.5: Zoho Calendar Showing the Follow-up Event Created for Report 6 (BUG-06 Fixed) 43
Figure 6.6: Action Taken Report Listing Four Records 44
Figure 7.1: Environments Page Showing Version 1.1 in Stage and Production (31 Oct 2026) 45
Figure 7.2: Publish Menu: Publish to Stage / Publish to Production 45
Figure 7.3: Administration: Settings → Users → Permissions 48
Figure 8.1: Scenario Step 1 – Severe Accident Report Entered with Status Reported 49
Figure 8.2: Scenario Step 2 – Notification E-mails Received (Test Road, Chennai and Salem Highway) 49
Figure 8.3: Scenario Step 3 – Stored Report 7 with Status Under Review 49
Figure 8.4: Scenario Step 4 – Dashboard with Open Accident Cases 50

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DS	Deluge Script (script ID used in this report)
ER	Entity–Relationship
FR / NFR	Functional / Non-Functional Requirement
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
UAT	User Acceptance Testing
UI	User Interface
UID	Unique Identifier
Zia	Zoho’s AI assistant

CHAPTER 1

INTRODUCTION

1.1 Background

Two-wheelers such as motorcycles, scooters, mopeds and electric bikes are a common means of daily travel, and their riders are directly exposed to the road. When a rider is involved in an accident, the facts that matter later are simple but easily lost: who the rider was, which vehicle was involved, when and where the accident happened, how severe it was, whether anyone was injured, whether a helmet was worn, and what was done afterwards (first aid, hospital referral, repair, an insurance claim or safety counselling).

In many small organisations, colleges, delivery fleets and community groups these facts are passed on by phone, written in a notebook or kept as photographs on a personal phone. Nobody has a single place to look at all the accidents, to see which cases are still open, or to see how many accidents were minor and how many were severe.

The Two-Wheeler Accident Reporting and Tracking System is a low-code application built on Zoho Creator to replace this practice. Riders and vehicles are registered once, every accident is recorded as a numbered report linked to its rider and vehicle, follow-up actions are recorded against the report, and a dashboard shows the state of all cases. The project was built as a general-purpose application and demonstrated with sample data (three riders, three vehicles and seven accident reports).

1.2 Problem Statement

Accidents involving two-wheeler riders are reported informally and are not tracked to closure. A simple digital system is required that stores rider and vehicle details, captures each accident in a structured form, alerts a responsible person when a new report arrives, treats severe accidents with priority, records the actions taken, and presents the overall position on one dashboard. The main pain points are:

- Accident details are scattered across notebooks, phone calls and phone galleries, so records are incomplete or lost.
- There is no quick way to see which cases are still open and which have been closed.
- Severe accidents are not treated differently from minor ones unless somebody notices them.
- Actions taken (first aid, hospital referral, repair, insurance) are not linked to the accident they belong to.

- No summary by severity or status is available, and every person who can see the data can see all of it.

1.3 Objectives

The project set out to achieve the following objectives:

- To design a data model of four linked forms (Rider, Vehicle, Accident Report and Action Taken) with lookup relationships, mandatory-field and no-duplicate validations.
- To automate the handling of a new accident report using Deluge scripting and form workflows: an e-mail notification that carries the severity, and automatic movement of Severe and Fatal reports to the status “Under Review”.
- To provide reports and a dashboard: list reports, an “Open Accident Cases” report that hides closed cases, and pivot charts of accidents by severity and by status, combined on an Accident Dashboard page built with the UI Builder.
- To integrate the application with Zoho Calendar through a Zoho Creator connection so that a follow-up event can be created for each new report.
- To control access through a role hierarchy (CEO, Safety Officer, Rider), permission sets and a data sharing rule, and to publish the application through the Stage and Production environments.

1.4 Scope of the Project

In scope. The implemented application contains four forms, reports built from them (list reports and two pivot charts), one page (Accident Dashboard), two form workflows containing three Deluge scripts, one Zoho Calendar connection, a three-level role hierarchy (CEO, Safety Officer, Rider), two custom permission sets (Safety Officer and Rider) and one data sharing rule. The application is used through a web browser.

Out of scope or not completed. The following were considered but are not part of the delivered application and are labelled as proposed wherever they appear: scheduled workflows, approval processes and Blueprints; Kanban or calendar reports; a customer portal or mobile configuration; automatic notification of the rider’s emergency contact; and field-level encryption. In addition, testing the application by signing in as users assigned to each role was done only informally with two test users (Section 5.8).

1.5 Justification for a Low-Code Approach

An accident register is a data-entry and workflow application: a few related forms, some reports, one or two calculations and a notification. These are exactly the building blocks that a

low-code application platform provides as configurable components. On Zoho Creator the four forms and their lookup relationships, the list and chart reports and the dashboard page were assembled with the form builder, the report builder and the UI Builder, and only the notification and the automatic status rule needed scripting in Deluge. Hosting, authentication, user management, record storage and report rendering are supplied by the platform, so the effort could be concentrated on the accident-handling logic instead of infrastructure. Published studies of low-code practice report that practitioners value the ease of learning and the speed of development of such platforms, while also reporting limits in customisation [1], [2].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2

LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The usual arrangement for recording two-wheeler accidents in small organisations is informal. After an accident the rider or a witness informs a family member, a workshop, an insurer or the police by phone or in person. Whoever receives the call notes the details on paper or in a register, photographs and documents stay in a personal phone, and follow-up on injury, repair or insurance is done by further phone calls. When an earlier case has to be found again, the record is either read from paper or the details are asked from the rider again. Figure 2.1 summarises this commonly followed practice. The description is a general account of informal accident handling and is not a survey of one named organisation.

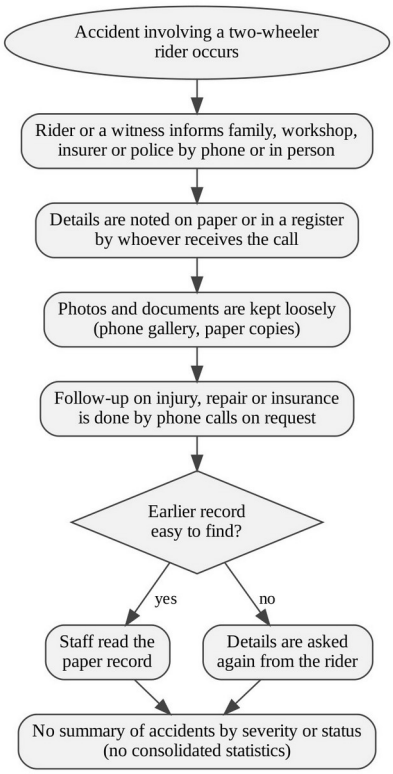


Figure 2.1: As-Is Process Flow of Informal Accident Reporting

2.2 Limitations of the Existing System

- **Scattered records.** Details are spread over paper, phone calls and personal phones, and are easily lost or duplicated.
- **No case tracking.** There is no status for a case, so open and closed cases cannot be told apart.

- **No priority for severe accidents.** A severe accident reaches the responsible person only if somebody chooses to pass it on.
- **Actions are not linked.** First aid, referrals, repairs and insurance claims are not tied to the report they belong to.
- **No summary.** The number of accidents by severity or by status cannot be seen without counting by hand.
- **No access control.** Whoever holds the notebook or the spreadsheet can see and change everything.

2.3 Low-Code versus Traditional Development

Table 2.1 compares a traditional pro-code implementation of the accident register with the low-code implementation on Zoho Creator. The comparison is qualitative. No development-time or cost measurements were recorded during the project, so none are claimed.

Table 2.1: Comparison of Low-Code and Traditional Development for the Accident Reporting Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Database design, back-end services, user interface and deployment pipeline must each be built and integrated.	Forms, reports and the dashboard are assembled in builders; only three short Deluge scripts were written.
Skills required	A front-end and a back-end language, SQL, hosting and security administration.	Familiarity with the platform and basic Deluge scripting.
Hosting and infrastructure	Servers or cloud services must be provisioned, secured and maintained.	Hosted and maintained by the platform.
Mobile application	A separate mobile client or responsive front end must be developed.	The builder offers desktop, tablet and phone preview toggles; a dedicated mobile configuration was not done.
Security and access control	Authentication, sessions and role logic must be implemented in code.	Roles, permission sets and data sharing rules are configured in Settings.
Maintenance and change	Changes need code changes, testing and redeployment.	Fields, reports and workflows are edited in the builders; scripts in the Deluge editor.
Customisation limits	Practically unlimited.	Bounded by platform components and Deluge capabilities (see Section 8.4).
Cost	Developer time plus infrastructure and licences.	Platform subscription; this project was built on a trial edition.

2.4 Literature Survey

Table 2.2 summarises the sources used. Two are published empirical studies of low-code practice and three are entries for the vendor documentation of the platform features used in this project. Only the points that the cited sources are known to cover are recorded; the bibliographic details of the two papers should be checked against the publisher pages before submission.

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, 2021 [1]	Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective	Empirical study of practitioners' discussions on low-code development, covering its advantages and its challenges.	Supports the choice of a low-code platform and the limitations discussion in Section 8.4.
2	M. A. Al Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider and A. Iqbal, 2021 [2]	An Empirical Study of Developer Discussions on Low-Code Software Development Challenges	Empirical study of developer questions about low-code platforms, grouped into topics of challenges.	Background for the limitations and workarounds discussed in Sections 8.4 and 8.5.
3	Zoho Corporation [3]	Zoho Creator Help	Documentation of forms, reports, pages, workflows, connections and user permissions.	Reference for Chapters 4 and 5.
4	Zoho Corporation [4]	Deluge Script reference	Documentation of the Deluge scripting language used in Creator workflows.	Reference for the scripts in Section 5.5 and Appendix C.
5	Zoho Corporation [3]	Zoho Creator Help – environments and user management	Documentation of the Stage and Production environments and of roles and permission sets.	Reference for Sections 5.8 and 7.1.

2.5 Emerging Trends in Low-Code Development

Three general trends are relevant to this project. First, low-code platforms increasingly embed AI assistance in their builders; the Zia assistant icon is visible on the Zoho Creator builder screens captured for this project, although no Zia feature is part of the delivered application. Second, platforms emphasise integration with the surrounding service ecosystem, which this project exercises through a Zoho Calendar connection. Third, as more business users build applications, governance concerns such as role design, data privacy settings and environment management become more important; this project applies them through permission sets, a role

hierarchy, personal-data flags and the Stage and Production environments. These trends are stated qualitatively and are not backed by market figures.

2.6 Proposed System

The proposed system is the Two-Wheeler Accident Reporting and Tracking System. Riders and vehicles are registered once. Each accident is recorded in an Accident Report form that is linked to a rider and a vehicle, receives an automatic report number and starts with the status Reported. A Deluge script moves Severe and Fatal reports straight to “Under Review”, and an e-mail is sent to the administrator with the severity in its subject. Actions taken are recorded against the report. A filtered report shows only the open cases, two pivot charts summarise severity and status, and the Accident Dashboard shows them together. Access is controlled through a role hierarchy and permission sets. The system therefore removes the scattered notes (Section 2.2, first point), gives every case a status, treats severe cases with priority, links actions to reports, summarises the data and restricts access.

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator	Owner of the Creator application; receives the notification e-mail.	Configure forms, reports, workflows, roles and permissions; register riders and vehicles; view the dashboard and all reports.
CEO (role)	Top level of the role hierarchy defined in Settings.	Oversight of the records of the roles below it in the hierarchy.
Safety Officer	Second level of the hierarchy; assigned the Safety Officer permission set.	View all accident reports, update their status, record actions taken, view the dashboard.
Rider	Third level of the hierarchy; assigned the Rider permission set.	Submit accident reports and view own records; cannot edit or delete.

3.2 Functional Requirements

Table 3.2 lists the functional requirements. FR-01 to FR-12 correspond to features built in the application (FR-11 is described in Section 5.7); FR-13 and FR-14 are proposed requirements that have not been implemented. The IDs are used again in the test cases of Chapter 6.

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall store rider details with Rider name and Phone number mandatory and a Licence Number that cannot be duplicated.	Administrator	High
FR-02	The system shall store vehicle details linked to a rider, with a mandatory, non-duplicate Registration Number.	Administrator	High
FR-03	The system shall allow an accident report to be submitted with a mandatory rider, vehicle, accident date and time, and location.	Rider, Safety Officer	High
FR-04	The system shall generate a report number automatically (Auto Number field).	System	High
FR-05	The system shall track each report in one of four statuses: Reported, Under Review, Action Taken, Closed (default Reported).	Safety Officer	High

FR ID	Requirement	Role(s)	Priority
FR-06	The system shall set the status of a Severe or Fatal report to Under Review automatically on submission.	System	High
FR-07	The system shall send an e-mail to the administrator when a report is submitted, with the severity in the subject.	System	High
FR-08	The system shall record actions taken (type, date, person, remarks) against an accident report.	Safety Officer	Medium
FR-09	The system shall provide a report listing only cases whose status is not Closed.	Safety Officer	Medium
FR-10	The system shall provide charts of accidents by severity and by status, and a dashboard that combines them with the open-case report.	Administrator, Safety Officer	Medium
FR-11	The system shall create a follow-up event in Zoho Calendar for a new report through a configured connection.	System	Medium
FR-12	The system shall restrict users through a role hierarchy, permission sets and a data sharing rule.	Administrator	Medium
FR-13	(Proposed) The system shall notify the rider's emergency contact for Severe and Fatal accidents.	System	Proposed
FR-14	(Proposed) The system shall remind the Safety Officer of cases that stay open too long, using a scheduled workflow.	System	Proposed

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Reports and the dashboard shall open without perceptible delay for the small record volumes of the demonstration. No measurement was recorded.
NFR-02	Security	Users shall reach only the forms, reports and pages allowed by their permission set; the emergency contact phone number shall be flagged as personal data.
NFR-03	Usability	An accident report shall be submitted from one form using drop-downs, radio buttons and lookups; the application shall run in a standard web browser.
NFR-04	Availability	The application relies on the availability of the Zoho Creator service; no separate hosting is maintained.
NFR-05	Maintainability	Business logic shall be confined to two named workflows so that it can be changed without redesigning the forms.

NFR ID	Category	Requirement
NFR-06	Auditability	Record changes are traceable only through the platform's own record information; a dedicated audit-trail configuration was not evidenced (Section 7.4).

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator, trial edition (India data centre, creator.zoho.in)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome (latest version)
Other Zoho services	Zoho Calendar (connection configured); e-mail sent through the Deluge sendmail task
API testing tool	Not used
Mobile	Not configured; the builder offers desktop, tablet and phone preview toggles
Client hardware	Any computer able to run a current web browser, with an internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases. The Rider submits accident reports. The Safety Officer updates the status of reports, records actions, and views the open cases and the dashboard. The Administrator registers riders and vehicles and manages roles and permissions, and also performs the Safety Officer use cases, as indicated by the dashed generalisation arrow, which is consistent with the administrator owning the application. The e-mail notification is sent to the administrator's mailbox when a report is submitted. Zoho Calendar appears as a secondary actor because it receives the follow-up event.

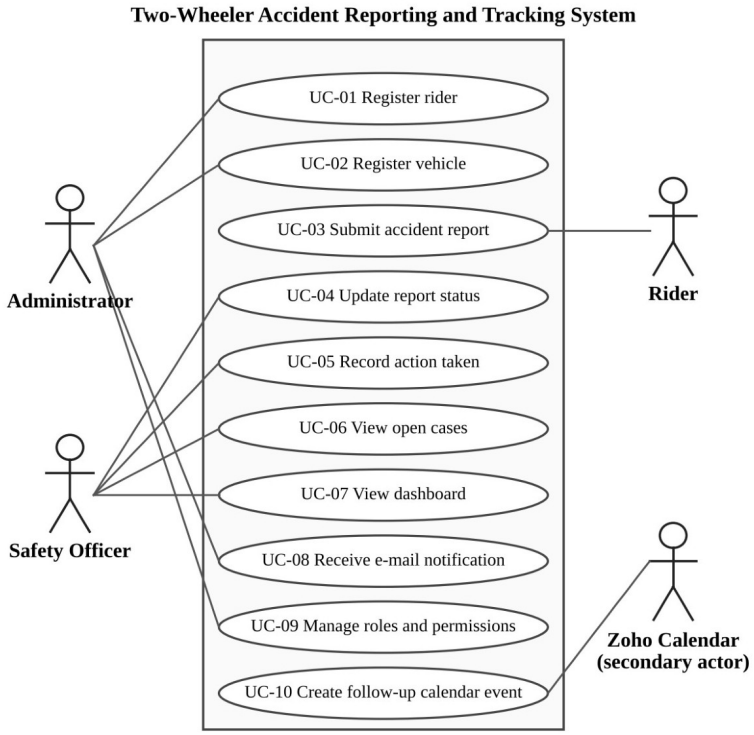


Figure 3.1: Use Case Diagram of the Two-Wheeler Accident Reporting and Tracking System

Tables 3.5 to 3.7 describe three key use cases in detail.

Table 3.5: Use Case Description – Submit Accident Report (UC-03)

Use Case ID	UC-03
Use Case Name	Submit Accident Report
Actor(s)	Rider (primary); Safety Officer; Administrator
Pre-condition	The rider and the vehicle are already registered in the Rider and Vehicle forms; the actor has access to the Accident Report form.
Main Flow	1. The actor opens Accident Reports → Accident Report. 2. The actor selects the Rider and the Vehicle from the lookups. 3. The actor enters the accident date and time, the location, the severity, injury and helmet details, and a description, and may attach a file. 4. The actor clicks Submit. 5. The system assigns the next report number; for Severe or Fatal severity it sets the status to Under Review, otherwise the status stays Reported. 6. The system e-mails the administrator and attempts to create a calendar event.
Alternate Flow	If a mandatory field (Rider, Vehicle, Accident Date and Time, Accident Location) is empty, the form does not submit and asks for the value.
Post-condition	A report record exists and appears in All Accident Reports and Open Accident Cases and in the dashboard charts.
Related FR	FR-03, FR-04, FR-05, FR-06, FR-07, FR-11

Table 3.6: Use Case Description – Update Report Status (UC-04)

Use Case ID	UC-04
Use Case Name	Update Report Status
Actor(s)	Safety Officer (primary); Administrator
Pre-condition	An accident report exists; the actor's permission set allows editing of the Accident Report form (Safety Officer: Edit on 2 reports).
Main Flow	1. The actor opens Accident Reports → Open Accident Cases. 2. The actor opens a record and clicks Edit. 3. The actor changes Status to Under Review, Action Taken or Closed. 4. The actor saves the record.
Alternate Flow	When a case is Closed it no longer appears in Open Accident Cases but remains in All Accident Reports.
Post-condition	The record shows the new status; the dashboard charts reflect the change.
Related FR	FR-05, FR-09, FR-10, FR-12

Table 3.7: Use Case Description – Record Action Taken (UC-05)

Use Case ID	UC-05
Use Case Name	Record Action Taken
Actor(s)	Safety Officer (primary); Administrator
Pre-condition	An accident report exists; the actor's permission set gives Access, View and Edit on the Action Taken form.
Main Flow	1. The actor opens Action Taken → Action Taken. 2. The actor chooses the Accident Report (the lookup shows the report number). 3. The actor chooses the Action Type and enters the Action Date, Action Taken By and Remarks. 4. The actor clicks Submit.
Alternate Flow	If the wrong report number was chosen, the actor opens the record in the Action Taken report and edits it.
Post-condition	The record appears in the Action Taken report. The report status is not changed automatically; the Safety Officer edits it (UC-04).
Related FR	FR-08

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The application runs entirely inside Zoho Creator; it has no custom front end, separate back end or external database. Figure 4.1 shows the architecture. Administrators, Safety Officers and Riders use the application through a web browser. The presentation layer consists of the four forms, the reports and the Accident Dashboard page. The logic layer consists of two form workflows on the Accident Report form that hold three Deluge scripts: DS-02 runs when the form is validated on submission, while DS-01 and DS-03 run after a successful submission. The data layer is the set of Rider, Vehicle, Accident Report and Action Taken records held by Creator and related through lookup fields. A role hierarchy, two permission sets and a data sharing rule govern access. DS-01 sends an e-mail to the administrator, and DS-03 calls Zoho Calendar through a Creator connection. The calendar path is drawn with dashed lines because the event could not be confirmed in testing (Section 5.7).

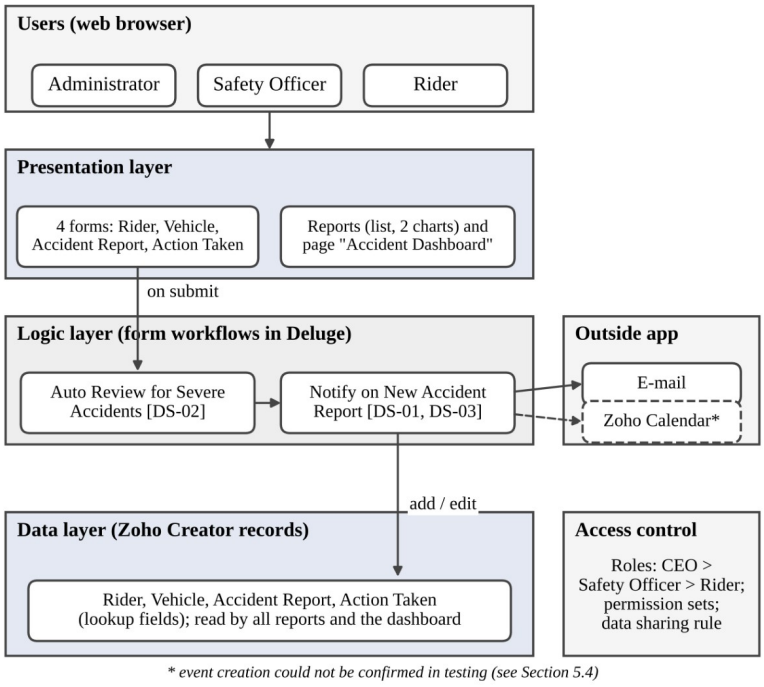


Figure 4.1: System Architecture of the Two-Wheeler Accident Reporting and Tracking System

4.2 Data Model

Zoho Creator stores data as records of forms. A form therefore plays the role of a table in a relational database, a field plays the role of a column, and a lookup field plays the role of a foreign key that points to a record of another form. The application uses four forms. Rider

holds the person’s details. Vehicle holds each vehicle and carries a Rider lookup, so one rider can own many vehicles. Accident Report holds each accident and carries both a Rider lookup and a Vehicle lookup. Action Taken holds the follow-up actions and carries an Accident Report lookup, so one accident can have many actions. Figure 4.2 shows the entities and relationships and Table 4.1 summarises them. No subform is used because every one-to-many relationship here is between records that are also useful on their own (a vehicle, an accident).

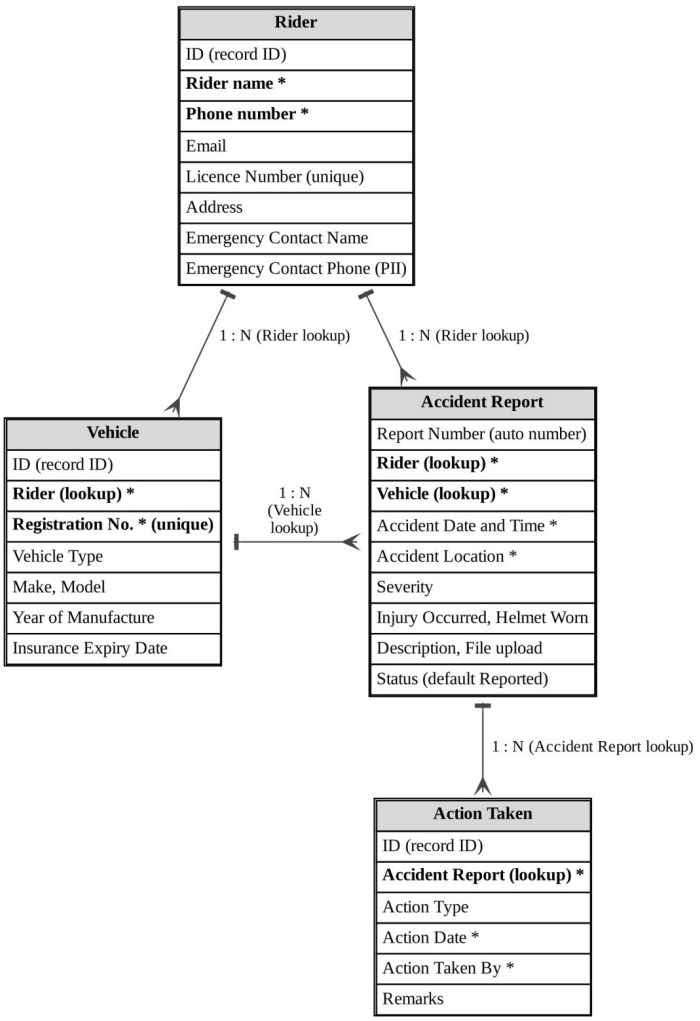


Figure 4.2: Entity–Relationship Diagram of the Application (* = mandatory)

Data consistency is supported in four ways. First, Rider, Vehicle and Accident Report values are chosen from existing records through lookups, so a report cannot name a rider or vehicle that does not exist. Second, Report Number is generated by an Auto Number field, so numbers are unique and sequential. Third, Severity, Status, Vehicle Type and Action Type are drop-downs with a fixed set of values. Fourth, Rider name and Phone number (Rider), Registration Number (Vehicle) and the key fields of the other forms are mandatory, while Licence Number and Registration Number do not allow duplicate values.

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Rider	Rider name, Phone number, Licence Number, Emergency Contact	Vehicle; Accident Report	One-to-Many (one rider, many vehicles and many reports)	Rider lookup in Vehicle and in Accident Report
Vehicle	Registration Number, Vehicle Type, Make, Model, Year, Insurance Expiry	Rider; Accident Report	Many-to-One to Rider; One-to-Many to Accident Report	Rider lookup in Vehicle; Vehicle lookup in Accident Report
Accident Report	Report Number, Accident Date and Time, Location, Severity, Status	Rider; Vehicle; Action Taken	Many-to-One to Rider and Vehicle; One-to-Many to Action Taken	Lookups to Rider and Vehicle; lookup in Action Taken
Action Taken	Action Type, Action Date, Action Taken By, Remarks	Accident Report	Many-to-One	Accident Report lookup

4.3 Form Design

Tables 4.2 to 4.5 specify the fields of the four forms. Field labels, types, mandatory flags and drop-down values were taken from the form-builder and live-form screenshots in Chapter 5. Field link names are shown exactly only where they are visible in the screenshots or used in the Deluge code (Emergency_Contact_Phone, Vehicle_Type, Status, Remarks, Accident_Location, Severity, Description); the other link names follow Creator's default convention of replacing spaces with underscores and should be confirmed in the builder.

Table 4.2: Field Specification – Rider Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Rider name	Rider_name	Name	Yes	—	First and last name
Phone number	Phone_number	Phone	Yes	Country-code selector (+91)	—
Email	Email	Email	No	—	—
Licence Number	Licence_Number	Single Line	No	No duplicate values	Unique per rider
Address	Address	Multi Line	No	—	—
Emergency Contact Name	Emergency_Contact_Name	Single Line	No	—	—
Emergency Contact Phone	Emergency_Contact_Phone	Phone	No	Contains personal data ticked	Flagged as PII (Figure 5.3)

Table 4.3: Field Specification – Vehicle Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Rider	Rider	Lookup (Rider)	Yes	—	Owner of the vehicle
Registration Number	Registration_Number	Single Line	Yes	No duplicate values	e.g. TN01AB1001
Vehicle Type	Vehicle_Type	Drop Down	No	—	Motorcycle, Scooter, Moped, Electric Bike
Make	Make	Single Line	No	—	—
Model	Model	Single Line	No	—	—
Year of Manufacture	Year_of_Manufacture	Number	No	Number format corrected (BUG-01)	—
Insurance Expiry Date	Insurance_Expiry_Date	Date	No	Format dd-MMM-yyyy	—

Table 4.4: Field Specification – Accident Report Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Report Number	Report_Number	Auto Number	—	System generated	Sequential, shown as the lookup display
Rider	Rider	Lookup (Rider)	Yes	—	—
Vehicle	Vehicle	Lookup (Vehicle)	Yes	—	—
Accident Date and Time	Accident_Date_and_Time	Date-Time	Yes	—	—
Accident Location	Accident_Location	Single Line	Yes	—	Used in e-mail text
Severity	Severity	Drop Down	No	—	Minor, Moderate, Severe, Fatal
Injury Occurred	Injury_Occurred	Radio	No	Yes / No	—
Helmet Worn	Helmet_Worn	Radio	No	Yes / No	—
Description	Description	Multi Line	No	—	Used in e-mail text
File upload	File_upload	File Upload	No	—	Photos or documents
Status	Status	Drop Down	No	Default: Reported	Reported, Under Review, Action Taken, Closed

Table 4.5: Field Specification – Action Taken Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Accident Report	Accident_Report	Lookup (Accident Report)	Yes	—	Shows the report number
Action Type	Action_Type	Drop Down	No	—	First Aid, Hospital Referral, Vehicle Repair, Insurance Claim Filed, Safety Counselling
Action Date	Action_Date	Date	Yes	—	—
Action Taken By	Action_Taken_By	Single Line	Yes	—	—
Remarks	Remarks	Multi Line	No	—	Free-text note

4.4 Process and Workflow Design

Accident report processing. Figure 4.3 shows what happens when an accident report is submitted. Before the record is saved, the workflow “Auto Review for Severe Accidents” runs a short script that sets the status to “Under Review” if the severity is Severe or Fatal. After the record is saved, the workflow “Notify on New Accident Report” runs: it sends an e-mail to the administrator whose subject contains the severity and then calls Zoho Calendar to create a follow-up event. The calendar step calls the Zoho Calendar API and creates an event that starts at the accident date and time and lasts one hour (Section 5.7).

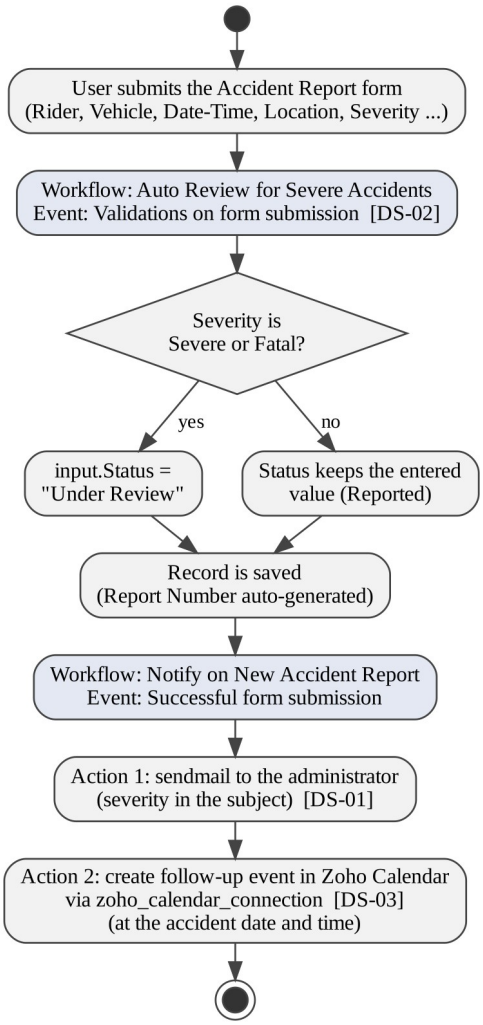


Figure 4.3: Workflow / Process Flow for Submitting an Accident Report

Status lifecycle. Figure 4.4 and Table 4.6 show the statuses. Reports start as Reported, or as Under Review when the severity is Severe or Fatal. All other changes are manual edits by the Safety Officer. Status is a drop-down, so the order shown is the intended order and is not enforced by the platform; no Blueprint or approval process is used.

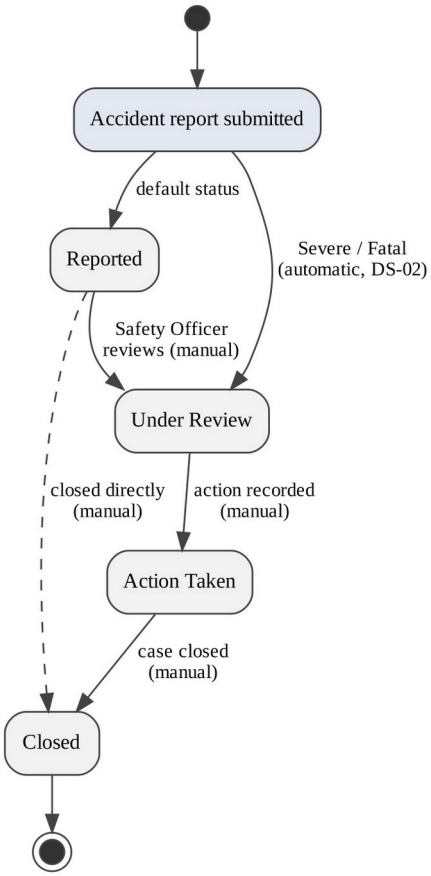


Figure 4.4: Status Lifecycle of an Accident Report

Table 4.6: Accident Report Status Values and Transitions

Status	Entered When	Changed By	Evidence
Reported	Report is submitted (default value).	Form default	Reports 1–6 in the list reports
Under Review	Severity is Severe or Fatal on submission, or the Safety Officer starts review.	DS-02 (automatic) or staff	Report 7 (Figure 8.3)
Action Taken	An action has been carried out for the report.	Safety Officer (manual)	Chart “Accidents by Status”
Closed	The case is finished.	Safety Officer (manual)	Report 1 closed; Open Accident Cases then shows 4 of 4

4.5 User Interface and Navigation Design

Figure 4.5 shows the navigation structure of the application menu. The menu has the sections Rider, Vehicles, Accident Reports and Action Taken, plus the page Accident Dashboard. Each section contains the form and the reports built from it; the Accident Reports section also holds the Open Accident Cases list and the two pivot charts. The Accident Dashboard combines the severity pie chart, the status bar chart and the Open Accident Cases report on one page. The

interface uses the platform's standard dark side menu and form layout, as seen in the screenshots of Chapter 5.

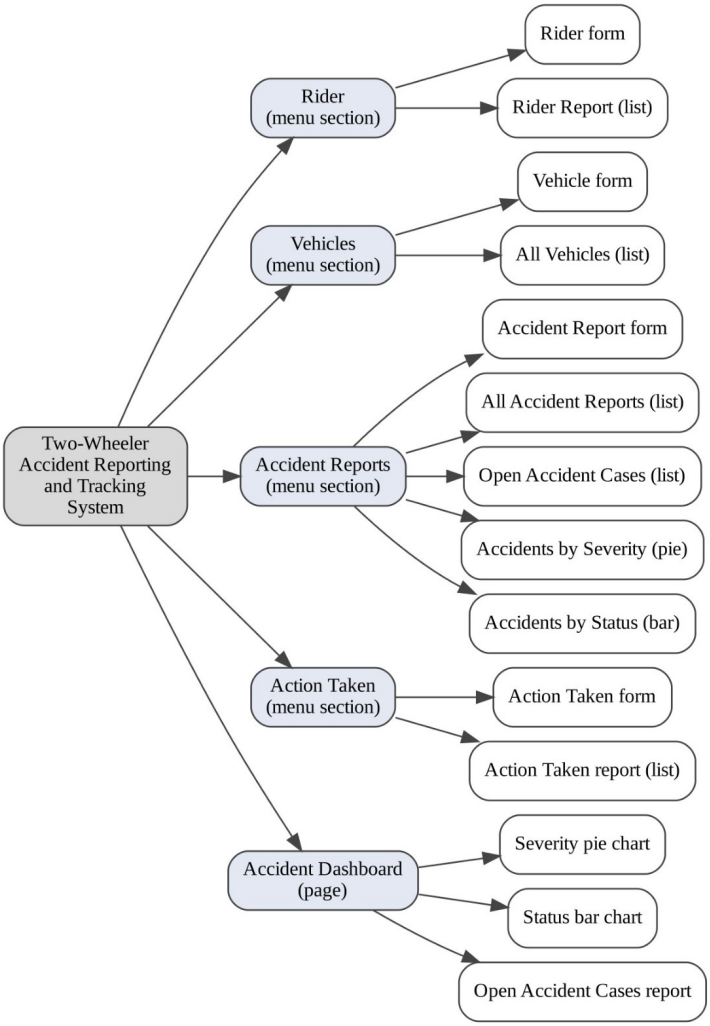


Figure 4.5: Navigation Structure of the Application

4.6 Access Control Design

Access is designed on three levels. Application access is by Zoho sign-in. A role hierarchy places users in a tree: CEO, then Safety Officer, then Rider. Permission sets define what each kind of user may do with each form, report and page, and a data sharing rule lets the Safety Officer role see the records of the Rider role. Table 4.7 records the design; A = Add (open the form), V = View, E = Edit, D = Delete, — = no access. The Administrator is the owner of the application and has full access.

Table 4.7: Role–Permission Matrix

Component	Administrator	Safety Officer	Rider
Form: Rider	A E V D	V	V (Rider Report)

Component	Administrator	Safety Officer	Rider
Form: Vehicle	A E V D	V	V (All Vehicles)
Form: Accident Report	A E V D	A V E	A V (All Accident Reports)
Form: Action Taken	A E V D	A V E	A V
Report: Open Accident Cases	V	V E	—
Page: Accident Dashboard	V	V	—

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

The application was created from scratch in a Zoho Creator account (India data centre) under the name “Two-Wheeler Accident Reporting and Tracking System”. The builder opens with the prompt “Start building your application by creating a form” (Figure 5.1). Four forms were created, and the menu was organised into the sections Rider, Vehicles, Accident Reports and Action Taken, with the page Accident Dashboard as a separate entry (Figure 5.2). The application header shows that the account was on a trial edition when the screenshots were taken. The application link name is two-wheeler-accident-reporting-and-tracking-system. The application address is https://mohammedsaajidkhanma202344.zohocreatorportal.in.

Table 5.1 summarises how the application is set up.

Table 5.1: Application Set-up Summary

Attribute	Details
Application name	Two-Wheeler Accident Reporting and Tracking System
Application link name	two-wheeler-accident-reporting-and-tracking-system
Data centre	India (creator.zoho.in)
Edition	Trial edition (shown in the builder header)
Deployed version	1.1 in Stage and in Production, published on 31 October 2026 (Section 7.1)
Live URL	https://mohammedsaajidkhanma202344.zohocreatorportal.in
Forms	4: Rider, Vehicle, Accident Report and Action Taken (Section 5.2)
Reports	7 reports (Table 5.3)
Pages	1: Accident Dashboard (Section 5.4)
Workflows	2 form workflows holding 3 Deluge scripts, DS-01 to DS-03 (Tables 5.5 and 5.6)
Integrations	Zoho Calendar through the connection zoho_calendar_connection; e-mail through sendmail (Table 5.7)

Attribute	Details
Roles and profiles	3 roles (CEO, Safety Officer, Rider), 2 custom permission sets, 1 data sharing rule (Section 5.8)
Account users	The administrator and two test users, one Rider and one Safety Officer (Section 5.8)

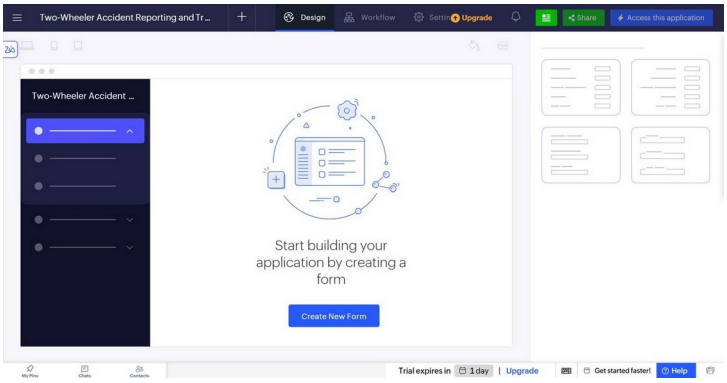


Figure 5.1: New Application in the Creator Builder

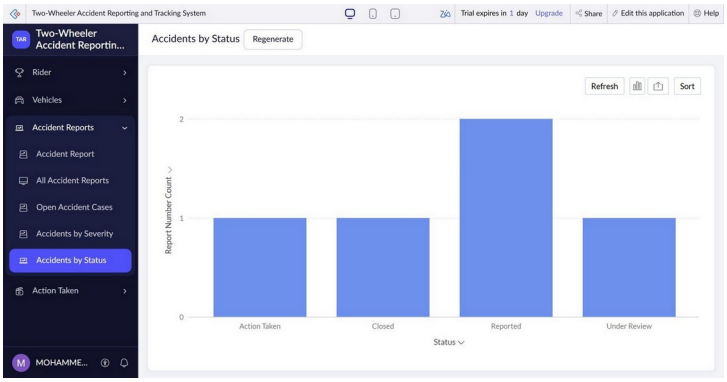


Figure 5.2: Application Menu with the Accident Reports Section Expanded

5.2 Forms and Data Capture

The application stores its data in four forms: Rider, Vehicle, Accident Report and Action Taken. Table 5.2 summarises them; the full field specification is given in Tables 4.2 to 4.5 of Chapter 4.

Table 5.2: Forms Implemented

Form	Main Fields	Mandatory Fields	Special Features
Rider	Rider name, Phone number, Email, Licence Number, Address, Emergency Contact Name and Phone	Rider name, Phone number	No duplicate Licence Number; country-code phone selector (+91); emergency contact phone flagged as personal data
Vehicle	Rider (lookup), Registration Number, Vehicle Type, Make, Model, Year of Manufacture, Insurance Expiry Date	Rider, Registration Number	Lookup to Rider; no duplicate Registration Number; Vehicle Type drop-down

Form	Main Fields	Mandatory Fields	Special Features
Accident Report	Rider, Vehicle, Accident Date and Time, Accident Location, Severity, Injury Occurred, Helmet Worn, Description, File upload, Status	Rider, Vehicle, Accident Date and Time, Accident Location	Auto-number Report Number; lookups to Rider and Vehicle; default Status Reported; 2 form workflows holding 3 scripts
Action Taken	Accident Report (lookup), Action Type, Action Date, Action Taken By, Remarks	Accident Report, Action Date, Action Taken By	Lookup to Accident Report; Action Type drop-down; no workflow

Rider form. Figure 5.3 shows the Rider form in the builder. Rider name (Name) and Phone number (Phone) are mandatory. The properties panel for Emergency Contact Phone shows the link name Emergency_Contact_Phone and the “Contains personal data (PII)” option ticked, while “Encrypt data” is not selected. Licence Number does not allow duplicate values.

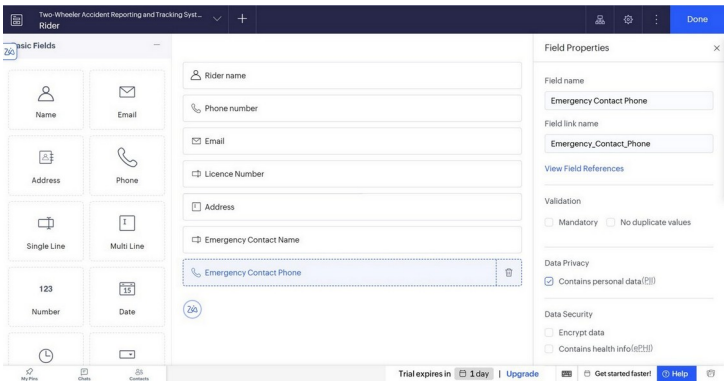


Figure 5.3: Rider Form in the Form Builder

Vehicle form. Figure 5.4 shows the Vehicle form. The Rider field is a lookup to the Rider form, Registration Number is mandatory with no duplicate values, and Vehicle Type is a drop-down with the choices Motorcycle, Scooter, Moped and Electric Bike.

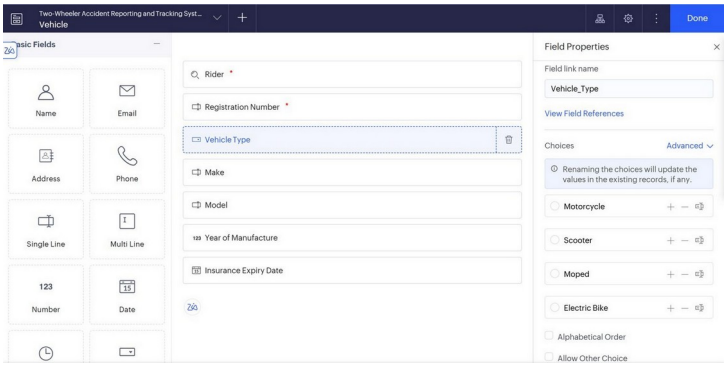


Figure 5.4: Vehicle Form in the Form Builder

Accident Report form. Figure 5.5 shows the lower part of the Accident Report form in the builder: Severity, Injury Occurred and Helmet Worn (radio buttons), Description, File upload and Status, whose choices are Reported, Under Review, Action Taken and Closed. Figure 5.8 later shows the same form in live mode.

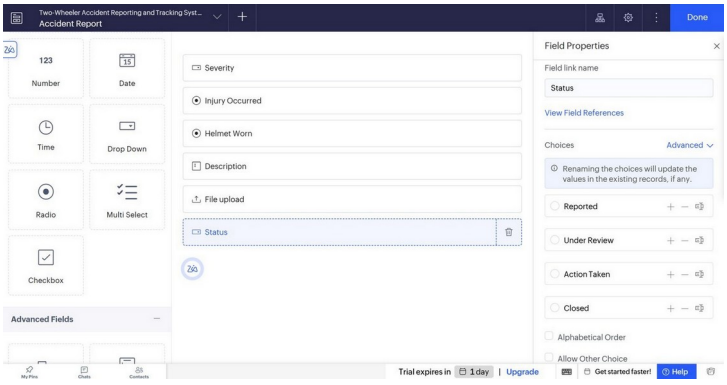


Figure 5.5: Accident Report Form in the Form Builder

Action Taken form. Figure 5.6 shows the Action Taken form with the lookup Accident Report, Action Type, Action Date, Action Taken By and Remarks.

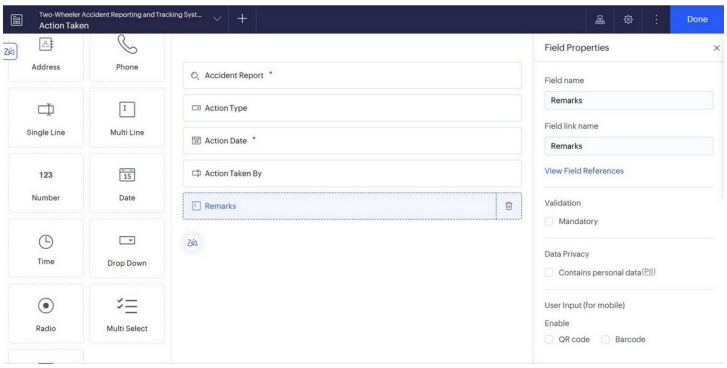


Figure 5.6: Action Taken Form in the Form Builder

Live mode. Figure 5.7 shows the Rider form in live mode with sample data. Figure 5.8 shows the Accident Report form with a Severe accident being entered; the Rider and Vehicle are chosen from lookups, and the Status field shows the default value Reported.

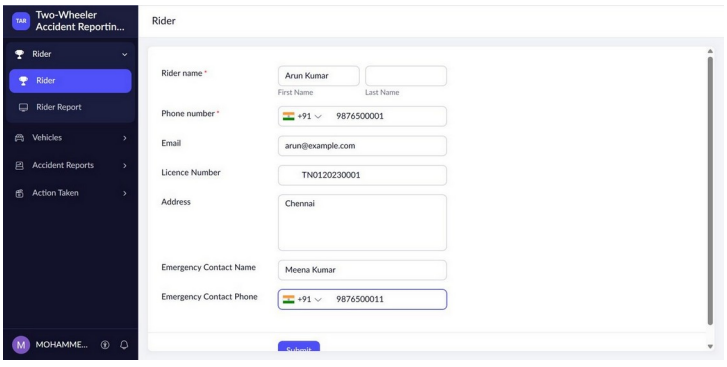


Figure 5.7: Rider Form in Live Mode (sample data)

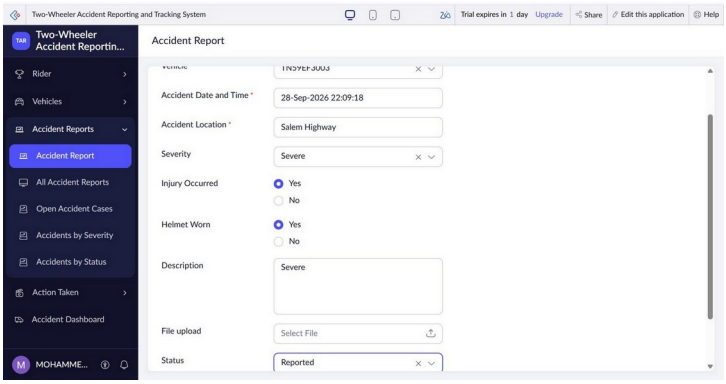


Figure 5.8: Accident Report Form in Live Mode

5.3 Reports

Creator generated a list report for each form (for example Rider Report, All Vehicles and All Accident Reports). Three further reports were created on the Accident Report form. **Open Accident Cases** is a list report whose filter is "Status not equal to Closed" (Figure 5.9); after report 1 was closed it showed 4 of 4 records (Figure 5.10), and clicking a row opens the detail view with the Edit, Duplicate and Delete actions (Figure 5.11). **Accidents by Severity** is a pivot chart (pie) with Severity on the X-axis and the count of Report Number on the Y-axis (Figure 5.12). **Accidents by Status** is a pivot chart (bar) with Status on the X-axis and the count of Report Number on the Y-axis (Figure 5.13). No grouping, sorting rule or custom button was configured, and Kanban, Calendar, Timeline and Map reports were not used.

Table 5.3: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Rider Report	List (auto-created)	Rider	None	Default Edit / Duplicate / Delete	Administrator; Rider (view)
All Vehicles	List (auto-created)	Vehicle	None	Default	Administrator; Safety Officer, Rider (view)
All Accident Reports	List (auto-created)	Accident Report	None	Default	Administrator; Safety Officer; Rider (view)
Open Accident Cases	List	Accident Report	Filter: Status not equal to Closed	Default Edit / Duplicate / Delete	Administrator; Safety Officer (view, edit)
Accidents by Severity	Pivot chart (pie)	Accident Report	X: Severity; Y: count of Report Number	None	Administrator
Accidents by Status	Pivot chart (bar)	Accident Report	X: Status; Y: count of Report Number	None	Administrator

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Action Taken report	List (auto-created)	Action Taken	None	Default	Administrator; Safety Officer; Rider (view)

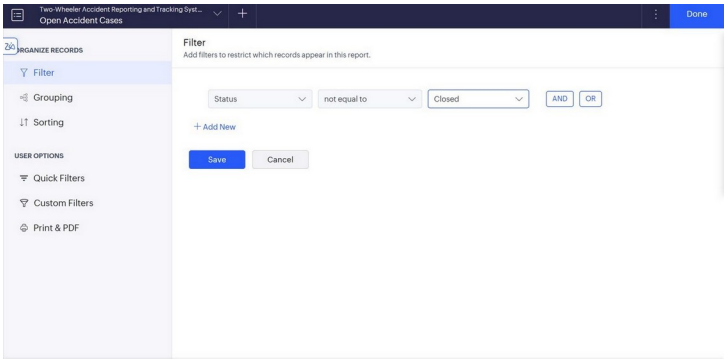


Figure 5.9: Filter Configuration of the Open Accident Cases Report

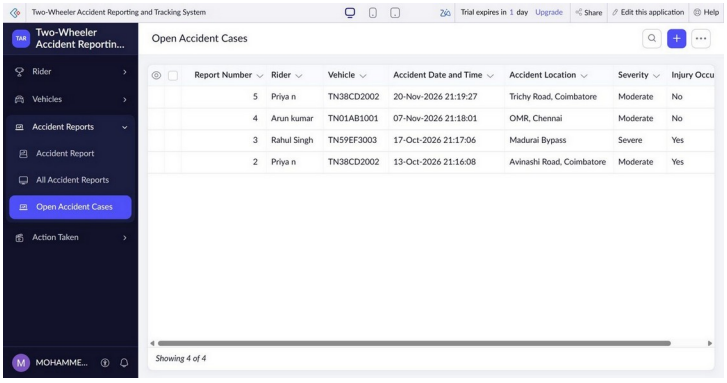


Figure 5.10: Open Accident Cases Report (4 of 4 records)

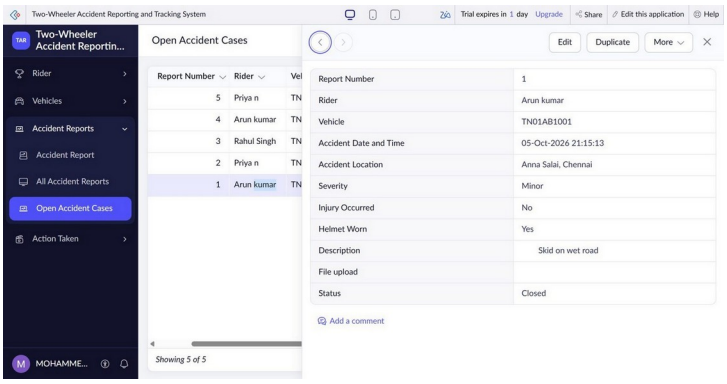


Figure 5.11: Detail View of a Record (report 1, status Closed)

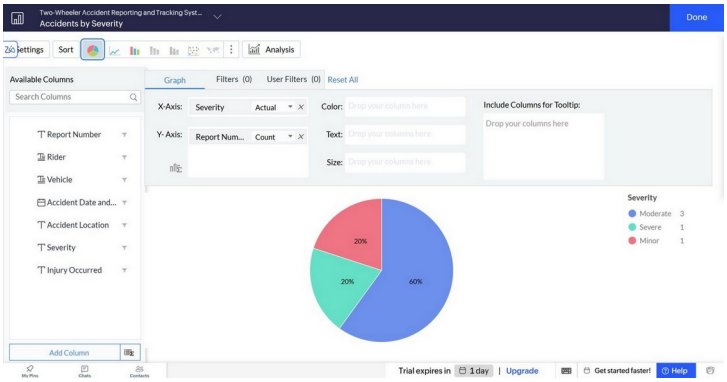


Figure 5.12: Accidents by Severity Pivot Chart in the Report Builder

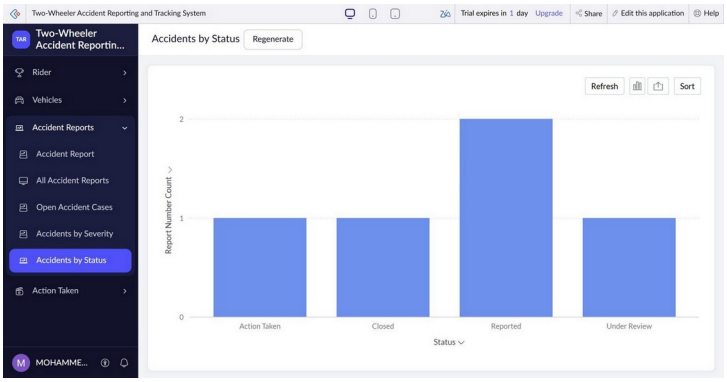


Figure 5.13: Accidents by Status Pivot Chart (live)

5.4 Pages and Dashboards

The page **Accident Dashboard** was built in the UI Builder, whose element palette offers Panel, Board, Chart, Gauge, Search, Form, Report, Snippets, Button, Widgets and Viewer (Figure 5.14). It is intended for the Administrator and the Safety Officer and contains three components: a Chart element (pie) on the Accident Report form showing the count by Severity, a Chart element (bar) showing the count by Status, and a Report element embedding Open Accident Cases. Two corrections were needed while building it: the pie chart first used “Actual data” and showed one slice per record, so it was changed to an aggregate count (BUG-04), and a stray Rider Report button was replaced with the Open Accident Cases report (BUG-05). Figures 5.15 and 5.16 show the finished page.

Table 5.4 lists the three components of the dashboard.

Table 5.4: Components of the Accident Dashboard

Component	Type	Data Source	Setting
Severity chart	Chart element (pie)	Accident Report	Aggregate count of Report Number by Severity (BUG-04 fix)
Status chart	Chart element (bar)	Accident Report	Aggregate count of Report Number by Status
Open Accident Cases	Report element (list)	Accident Report	Filter: Status not equal to Closed

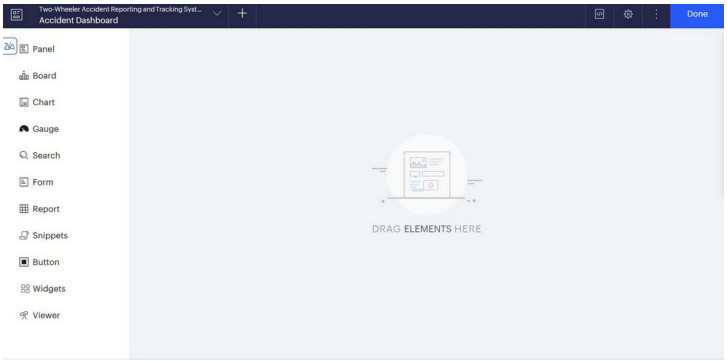


Figure 5.14: UI Builder Element Palette of the Accident Dashboard Page

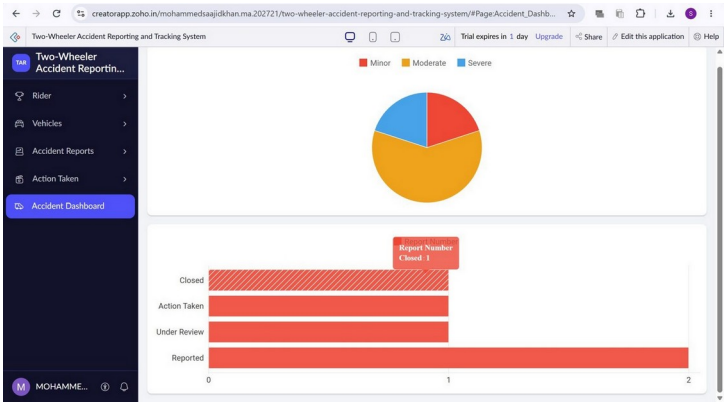


Figure 5.15: Accident Dashboard – Severity and Status Charts

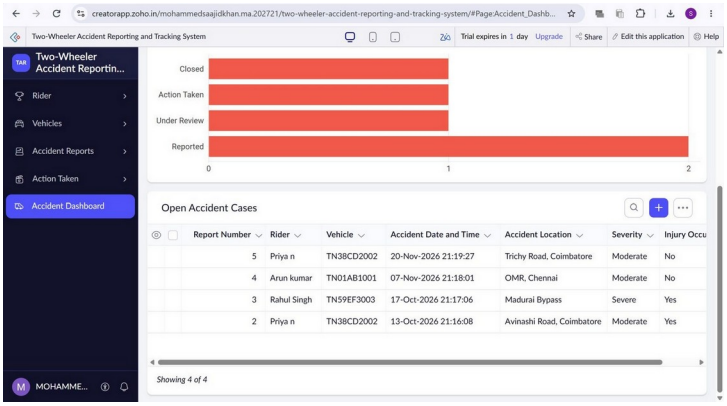


Figure 5.16: Accident Dashboard – Status Chart and Open Accident Cases Report

5.5 Deluge Scripting

Three Deluge scripts were written. They are configured as actions inside two form workflows on the Accident Report form. Table 5.5 lists them and the short snippets below show the logic; the complete code with comments is in Appendix C.

Table 5.5: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Accident Report → Created → Successful form submission (workflow Notify on New Accident Report, action 1)	E-mail the administrator when a report is submitted	sendmail, zoho.adminuserid, input.<field>, string concatenation
DS-02	Accident Report → Created → Validations on form submission (workflow Auto Review for Severe Accidents)	Set Status to Under Review for Severe or Fatal reports	if, logical OR, assignment to input.Status
DS-03	Same workflow as DS-01, action 2	Create a one-hour follow-up event in Zoho Calendar at the accident date and time	Map, put, subMinutes, toString, invokeurl, info

Code 5.1: Notification script (DS-01)

```
// DS-01 | Accident Report | Successful form submission
sendmail
[
from :zoho.adminuserid
to :zoho.adminuserid
subject :"New Accident Report Submitted - " + input.Severity
message :"A new accident report was submitted. Location: " +
input.Accident_Location + ". Severity: " + input.Severity + ".
Details: " + input.Description
]
```

Code 5.2: Automatic status script (DS-02)

```
// DS-02 | Accident Report | Validations on form submission
if(input.Severity == "Severe" || input.Severity == "Fatal")
{
input.Status = "Under Review";
}
```

DS-01 builds the e-mail subject and message from the values typed in the form. It was tested twice and the e-mail arrived from notifications@trial.zohocreatormail.com with the subject “New Accident Report Submitted – Severe” (Figure 8.2). Because the sender and the recipient are both the administrator’s address, the e-mail is delivered to the administrator only; sending to other users is not demonstrated. **DS-02** runs before the record is saved and changes the Status value that is about to be stored, so a Severe or Fatal report is saved as Under Review even if the form showed Reported (Figures 8.1 and 8.3). **DS-03** is described in Section 5.7. Standalone Deluge functions, fetch statements, loops and schedules were not needed for this application and were not used.

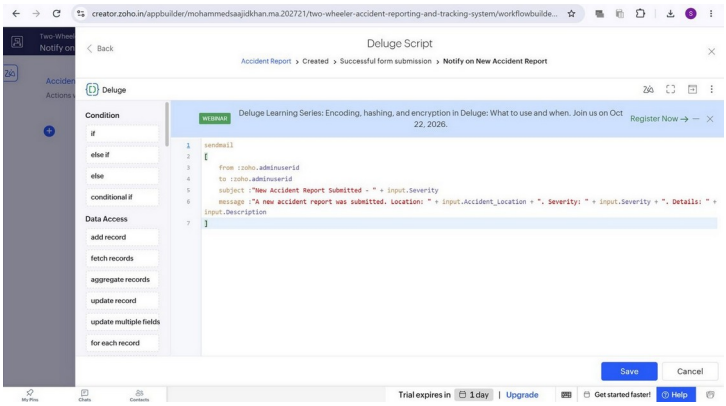


Figure 5.17: Deluge Editor with the Notification Script (DS-01)

5.6 Workflows and Automation

Only form workflows were used. The Workflow tab also offers Schedules, Approvals, Payments, Blueprints, Batch workflows and Functions; these were not needed for the delivered scope (see Section 9.2 for how they could be used). Creating a workflow starts from the "Run workflow on any event in the form" screen, where the form, the record event (Created), the form event and the workflow name are chosen (Figures 5.18 and 5.19).

Table 5.6: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Auto Review for Severe Accidents	Form workflow	Accident Report → Created → Validations on form submission	Severity is Severe or Fatal (inside the script)	Deluge script sets Status = Under Review	DS-02
Notify on New Accident Report	Form workflow	Accident Report → Created → Successful form submission	Execute without condition	Action 1: Deluge script (e-mail); Action 2: Deluge script (calendar event)	DS-01, DS-03

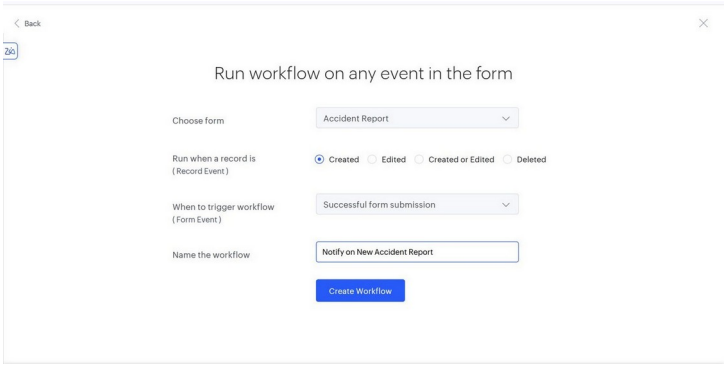


Figure 5.18: Creating the Workflow "Notify on New Accident Report"

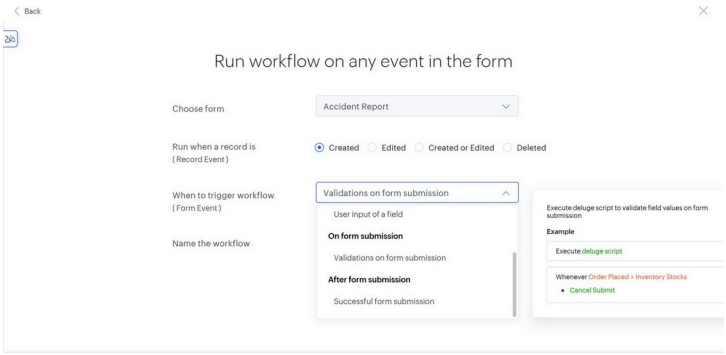


Figure 5.19: Choosing the “Validations on form submission” Event

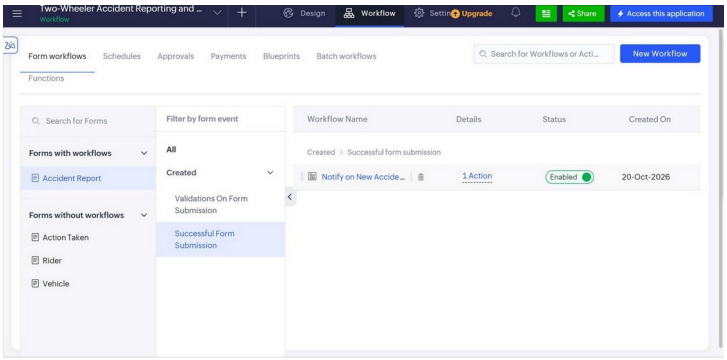


Figure 5.20: Workflow “Notify on New Accident Report” (Enabled)

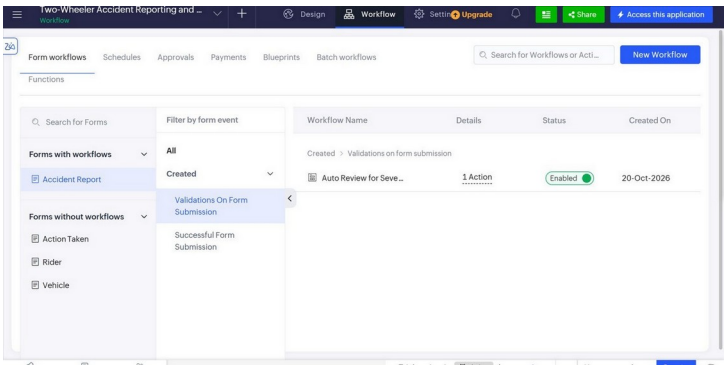


Figure 5.21: Workflow “Auto Review for Severe Accidents” (Enabled)

5.7 Integrations

Zoho Calendar. A connection named “Zoho Calendar Connection” with the link name zoho_calendar_connection exists under Microservices → Connections and shows the status Connected (Figure 5.22). Its permissions are “Create Calendar, Create Event, Get all Calendar”, the authorization is the same for all environments, and the owner is the Administrator. Connection access was switched on for this application (Figure 5.23). The calendar identifier needed by the script was taken from the Zoho Calendar settings (My Calendars) (Figure 5.25).

In the Deluge editor, the “zoho integration” block offers the service Zoho Calendar and the function createEvent with the parameters Calendar_uid (string), EventDetails (map) and

Connection_linkname (Figure 5.24). Script DS-03 was first written to use this call; it was later replaced by a direct call to the Calendar API (see the result below).

Table 5.7: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Calendar follow-up	Zoho Calendar – events API (POST)	Creator connection zoho_calendar_c onnection + Deluge invokeurl	Zoho connection (authorised by the Administrator)	Event title with the accident location, starting at the accident date and time and lasting one hour. **Status: working; the event was created in testing (Fig. 6.5).**
E-mail notification	Deluge sendmail	Built-in Deluge task (no connection)	Application administrator	Subject and text built from the report; see DS-01.

Code 5.3: Calendar integration script (DS-03, final version)

```
// DS-03 | Notify on New Accident Report | action 2 (final version)
eventDetails = Map();
eventDetails.put("title", "Follow-up: accident at " +
input.Accident_Location);
startUTC = input.Accident_Date_and_Time.subMinutes(330);
endUTC = input.Accident_Date_and_Time.subMinutes(270);
dateTime = Map();
dateTime.put("timezone", "Asia/Calcutta");
dateTime.put("start", startUTC.toString("yyyyMMdd") + "T" +
startUTC.toString("HHmmss") + "Z");
dateTime.put("end", endUTC.toString("yyyyMMdd") + "T" +
endUTC.toString("HHmmss") + "Z");
eventDetails.put("dateandtime", dateTime);
params = Map();
params.put("eventdata", eventDetails.toString());
eventResponse = invokeurl
[
url:
"https://calendar.zoho.in/api/v1/calendars/576ca0573f794b6995bba3d2
c916f96e/events"
type: POST
parameters: params
connection: "zoho_calendar_connection"
];
info eventResponse;
```

Result. The first version of DS-03 used the built-in call `zoho.calendar.createEvent` [5] with a start time on the next day, and no event appeared (BUG-06). To see the response, a `sendmail` statement was added that e-mailed `eventResponse.toString()` to the administrator. The response was the error code LESS_THAN_MIN_OCCURANCE. Changing the times to UTC gave the same error. A GET request with `invokeurl` to the

calendars endpoint returned the calendar with the same identifier, so the connection and the identifier were correct. The built-in call was therefore replaced by a POST request with `invokeurl` to the events endpoint of the Zoho Calendar API, sending the event as JSON in the parameter `eventdata`, and this created the event: the response contained the status “added”. The final script starts the event at the accident date and time (converted from India time to UTC) and ends it one hour later. For report 6, whose accident time was 02-Dec-2026 01:34:35, the event appeared on 2 December from 1:34 to 2:34 am (Figure 6.5). Why the built-in `createEvent` call returned the error was not determined.

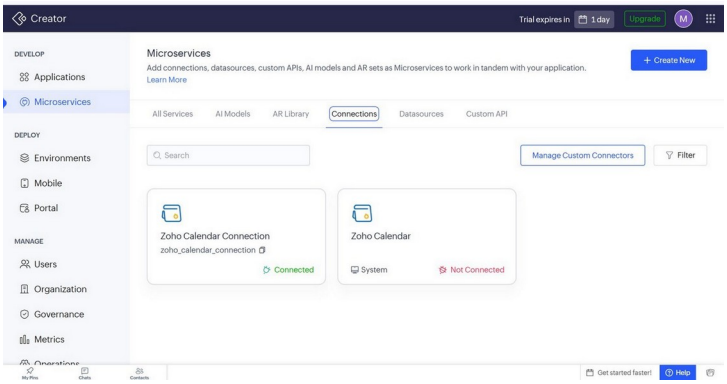


Figure 5.22: Connections List Showing Zoho Calendar Connection (Connected)

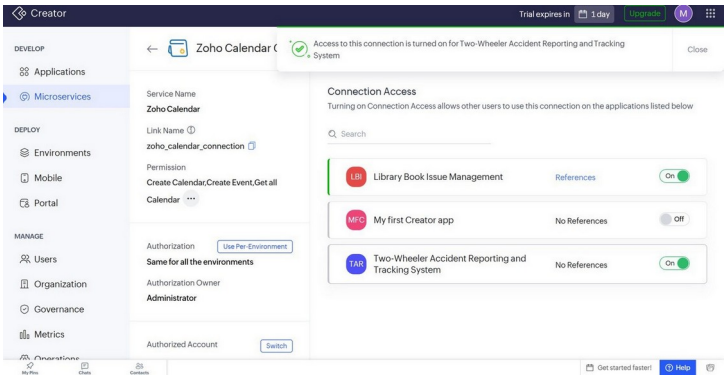


Figure 5.23: Connection Details and Access Turned On for the Application

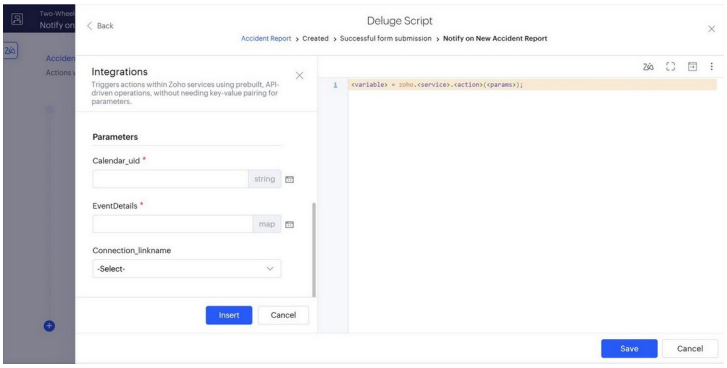


Figure 5.24: Deluge “zoho integration” Block for Zoho Calendar → createEvent

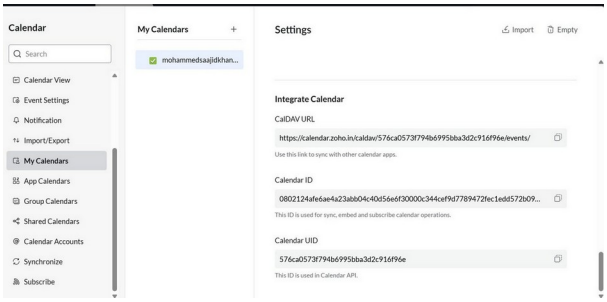


Figure 5.25: Zoho Calendar Settings Showing the Calendar UID

5.8 User Management and Role-Based Access

Under Settings → Users, the Permissions tab lists the built-in sets Read and Write ("This profile will have read / write permission for all components"). Two custom permission sets were added (Figure 5.26). **Safety Officer** ("Can view all accident reports and update their status") gives access to Accident Report with View on 4 reports and Edit on 2 reports (All Accident Reports and Open Accident Cases), Action Taken with Access, View and Edit, Rider and Vehicle with Access and View, and access to the Accident Dashboard (Figure 5.27). **Rider** ("Can submit accident reports and view their own records") gives Access and View on Accident Report (All Accident Reports), Action Taken, Rider (Rider Report) and Vehicle (All Vehicles), with no Edit or Delete and no access to the dashboard (Figure 5.28).

The Roles tab holds the hierarchy CEO → Safety Officer → Rider (Figure 5.29). The Data Sharing tab holds the rule "Share Rider Reports with Safety Officer" on the Accident Report form, sharing records from the role Rider to the role Safety Officer with the level of access Read Only (Figure 5.30).

Table 5.8 records the data sharing rule.

Table 5.8: Data Sharing Rules

Rule Name	Form	Shared From	Shared To	Level of Access
Share Rider Reports with Safety Officer	Accident Report	Role: Rider	Role: Safety Officer	Read Only

What was tried with users. Two test users were invited from the Users page and assigned to the roles: one Rider and one Safety Officer. Both could sign in and open the application. An invitation to a third address stayed at "Invited" and could not be accepted, and adding it again gave a Zoho error, so it was not used. The administrator's view and the Safety Officer's view differed as designed: the administrator sees four reports and the App Preferences menu (Figure 5.31), while the Safety Officer has no App Preferences (Figure 5.32). At first the Safety Officer saw "No Data Available" (0 of 0) in All Accident Reports, because the data sharing rule shares only the records created by users with the Rider role, and the four existing

reports had been created by the administrator. After a report was submitted with the Rider account, the Safety Officer account could see it. The Safety Officer could also open and submit the Rider form, because the Safety Officer permission set has Access to Rider; it has no Edit or Delete there.

What is not done. The checks above were informal and not a full test of every role: for example, test case TC-15 (a Rider cannot edit reports) was not run. No record-level criteria such as “Added User == zoho.loginuser” were configured. The note that the Rider permission set shows “their own records” describes the intended use; with View on All Accident Reports the set as saved is not restricted to own records, so a Rider can see other riders’ reports.

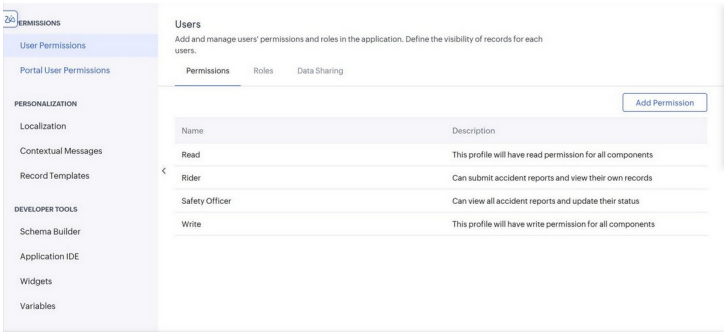


Figure 5.26: Permission Sets: Read, Rider, Safety Officer and Write

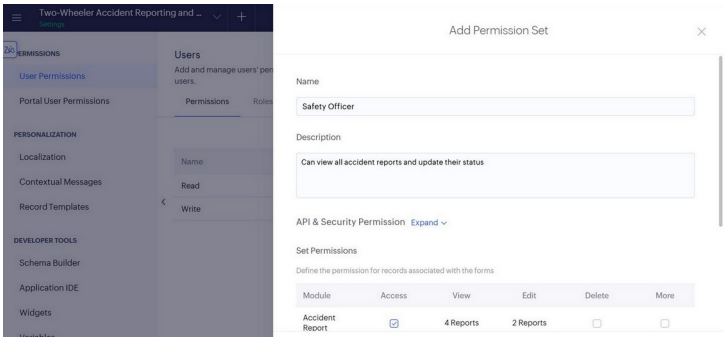


Figure 5.27: Safety Officer Permission Set

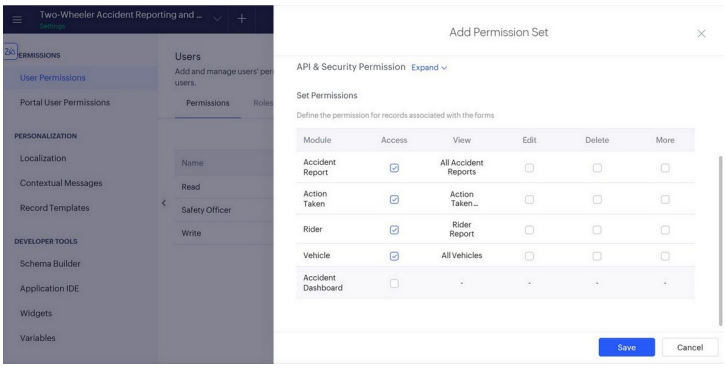


Figure 5.28: Rider Permission Set

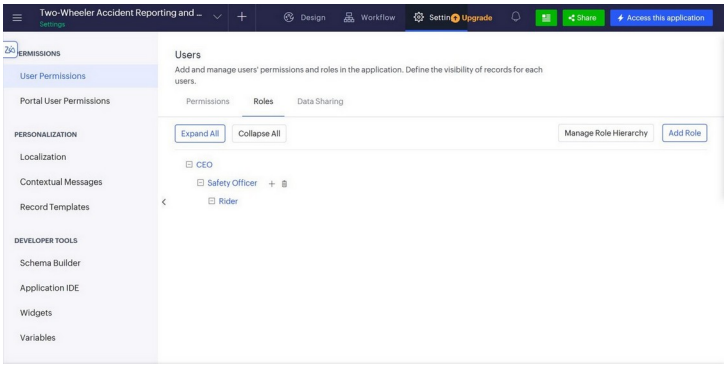


Figure 5.29: Role Hierarchy: CEO → Safety Officer → Rider

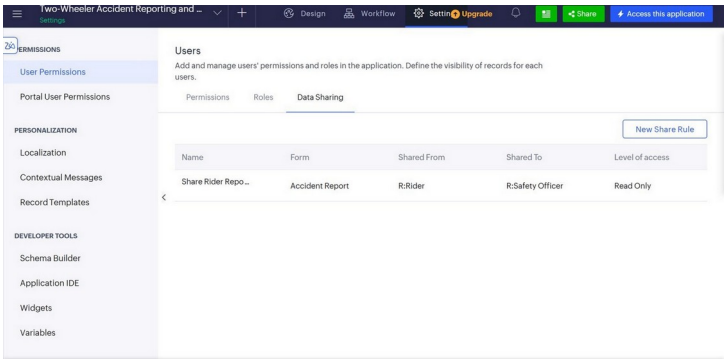


Figure 5.30: Data Sharing Rule “Share Rider Reports with Safety Officer”

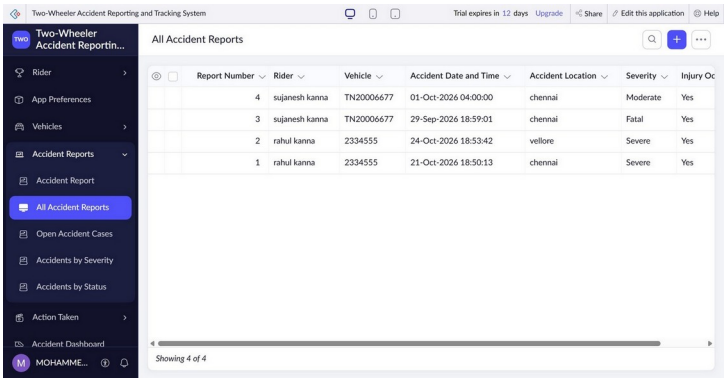


Figure 5.31: All Accident Reports Seen by the Administrator (4 of 4 Records)

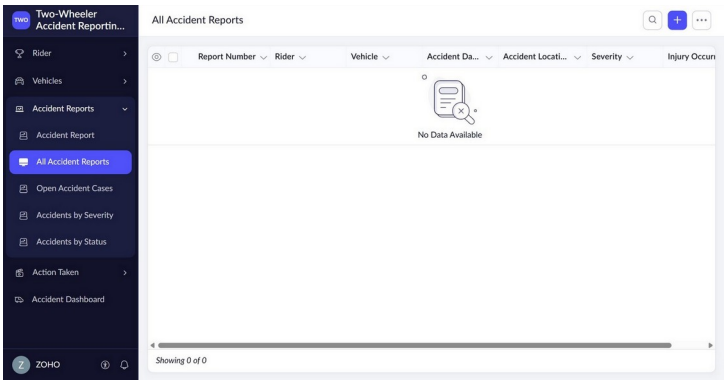


Figure 5.32: All Accident Reports Seen by the Safety Officer Before Any Rider Report (0 of 0)

5.9 AI and Analytics Features

Analytics. The analytics in the application are the two pivot charts (Section 5.3) and the two dashboard charts (Section 5.4), which count records by severity and by status. Zoho Analytics was not integrated.

AI features of the platform. No Zia or other AI feature of Zoho Creator (AI fields, prediction, OCR, keyword extraction, sentiment analysis or the AI application builder) is part of the delivered application.

AI assistance during the project. An AI assistant (Claude, by Anthropic) was used to guide the step-by-step construction of the application, to suggest the Deluge scripts DS-01 to DS-03 and the settings of the reports and permission sets, and to help draft this report from the screenshots. All configuration was carried out in Zoho Creator by the student, and the results were checked against the live application as shown in Chapter 6.

5.10 Pro-Code Extension

No pro-code extension (Zoho Catalyst, custom functions, JavaScript widgets or the Creator REST API) was used (the calendar script does call the Zoho Calendar API with `invokeurl` from Deluge, which is an integration call inside a low-code script and not a separate component), because the required behaviour (storing records, a notification, an automatic status rule and charts) was covered by forms, workflows and short Deluge scripts. A future extension could expose the Accident Report form through the Creator REST API so that a mobile app or a roadside kiosk could submit reports, or use a Catalyst function for heavier processing such as location-based accident hot-spot analysis.

5.11 Mobile and Portal Access

The builder offers desktop, tablet and phone preview toggles, but a dedicated mobile configuration, the Zoho Creator mobile app and a customer portal were not set up or tested. Letting riders submit reports from their phones through a portal is listed as a future enhancement (Section 9.2).

CHAPTER 6

TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done on the live application in Zoho Creator while it was being built. **Unit testing** checked each form by entering sample data (three riders, three vehicles and seven accident reports) and each report and chart by comparing it with the records. **Integration testing** checked the two workflows by submitting accident reports and observing the e-mail and the stored status. **Role-based testing** was done only informally: two test users (a Rider and a Safety Officer) signed in and the effect of the sharing rule was observed (Section 5.8), but the test cases for every role were not run. **User acceptance testing** with other users has not been recorded yet (Section 6.5). Two sample accident dates entered in November 2026 lie in the future of the entry date and are test data; the form has no validation that prevents future dates.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Submit Rider form with mandatory fields	Name “Arun Kumar”, phone +91 9876500001, licence TN0120230001	Record saved	Saved; sample riders listed in the Rider Report (Fig. 5.7)	Pass
TC-02	FR-01	Duplicate licence number rejected	Enter an existing Licence Number	Validation error	Not executed	Not run
TC-03	FR-02	Submit Vehicle with a Rider lookup	Rider “Arun kumar”, TN01AB1001, Motorcycle, 2021	Record saved	Saved after the Year format fix (BUG-01)	Pass
TC-04	FR-02	Duplicate registration number rejected	Enter TN01AB1001 again	Validation error	Not executed	Not run
TC-05	FR-03, FR-04	Submit Accident Report with all mandatory fields	Rider, Vehicle, date-time, location	Saved with next report number	Reports 1–7 saved with numbers 1–7	Pass
TC-06	FR-03	Mandatory fields left empty	Submit with Accident Location empty	Form asks for the value	Not executed	Not run
TC-07	FR-05	Default status	Submit a Minor report	Status = Reported	Reports 1–5 were saved as Reported (record detail, Fig. 5.11 shows report 1 later changed to Closed)	Pass
TC-08	FR-06	Severe report moves to Under Review	Submit Severe report 7 with Status Reported	Saved as Under Review	Report 7 shows Under Review (Fig. 8.3)	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-09	FR-07	Notification e-mail	Submit Severe reports at Test Road, Chennai and Salem Highway	E-mail with severity in subject	Two e-mails received, subject "… - Severe" (Fig. 8.2)	Pass
TC-10	FR-09	Open cases hide closed cases	Close report 1; open Open Accident Cases	Closed report not listed	4 of 4 records shown (Fig. 5.10)	Pass
TC-11	FR-10	Charts count records	Open the pivot charts and the dashboard	Counts match records	Pie by Severity and bar by Status match (Fig. 5.13, 5.15)	Pass
TC-12	FR-10	Dashboard shows charts and open cases	Open Accident Dashboard	Two charts and one report	Shown (Fig. 5.15, 5.16)	Pass
TC-13	FR-11	Calendar event created for a new report	Submit a report after saving DS-03	Event in Zoho Calendar	Event created at the accident date and time after the BUG-06 fix (Fig. 6.5)	Pass
TC-14	FR-12	Permission sets and sharing rule can be saved	Create Safety Officer, Rider sets and the sharing rule	Saved and listed	Listed (Fig. 5.26–5.30)	Pass
TC-15	FR-12	Rider user cannot edit reports	Sign in as a Rider user and try to edit	Edit not offered	Not executed	Not run
TC-16	FR-08	Record an Action Taken against a report	Choose report number, action type, date, person	Record saved	Four Action Taken records are listed in the Action Taken report (Fig. 6.6)	Pass

Table 6.2 summarises the 16 test cases of Table 6.1. No test case failed. The test cases that were not run are the duplicate and mandatory-field validations (TC-02, TC-04, TC-06) and the Rider edit test (TC-15), which need a separate test run.

Table 6.2: Summary of Test Results

Result	Count	Test Cases	Comment
Pass	12	TC-01, TC-03, TC-05, TC-07, TC-08, TC-09, TC-10, TC-11, TC-12, TC-13, TC-14, TC-16	Behaved as expected in the live application.
Fail	0	—	No test case failed after the fixes.
Not run	4	TC-02, TC-04, TC-06, TC-15	Validation and role tests that were not carried out.

6.3 Debugging Deluge Scripts

Debugging relied on four techniques: the error message shown by a form on submission, the behaviour of the live records after a test submission (for DS-02, comparing the Status typed in the form with the Status stored), the e-mail actually received (for DS-01), and the `info` statement at the end of DS-03, which writes the response of the calendar call to the log. The

workflow list shows each workflow as Enabled, which confirms that the scripts saved without syntax errors.

A real bug. Report 7 was submitted with Status “Reported” and Severity “Severe”. Because DS-02 runs on “Validations on form submission”, the saved record shows Under Review (Figures 8.1 and 8.3). An earlier idea to set the status in the “Successful form submission” event would have needed a second update of an already saved record; running the script before saving avoids it.

BUG-06 (calendar event). The output of `info eventResponse` was hard to find, so the response was e-mailed to the administrator instead. It showed the error LESS_THAN_MIN_OCCURANCE, with both a local-time and a UTC format. A list of calendars confirmed the identifier, and a direct `invokeurl` call to the Calendar API then created the event (Section 5.7, Figure 6.5). The reason why the built-in `createEvent` call failed was not determined. Figure 6.1 shows the first saved script and Figure 6.2 shows a form error message that was debugged.

Table 6.3 lists the symptoms that were traced with these techniques and what each trace showed.

Table 6.3: Debugging Observations

No.	Symptom	Technique Used	Finding	Defect
1	Form message “Input format does not match the required format ‘#######’” for Year of Manufacture 2021	Read the error message shown on submission	The Number field format did not accept a four-digit year	BUG-01
2	Report 7 typed as Reported was stored as Under Review	Compared the status typed in the form with the stored status	DS-02 runs on “Validations on form submission” and sets the status before the record is saved	— (as designed)
3	Open Accident Cases listed a closed report (5 of 5 records)	Compared the list with the record statuses	The filter “Status not equal to Closed” had not been applied	BUG-03
4	Dashboard pie chart showed one slice for every record	Compared the chart with the record list	The value type was “Actual data” and not an aggregate count	BUG-04
5	No calendar event after a report was submitted	E-mailed the response of the call to the administrator, then listed the calendars with invokeurl	createEvent returned LESS_THAN_MIN_OCCU RANCE although the calendar identifier and the connection were correct; an invokeurl POST created the event	BUG-06
6	The Safety Officer saw “No Data Available” while the administrator saw four reports	Compared the two views and the data sharing rule	The rule shares only records created by users with the Rider role	— (as designed)

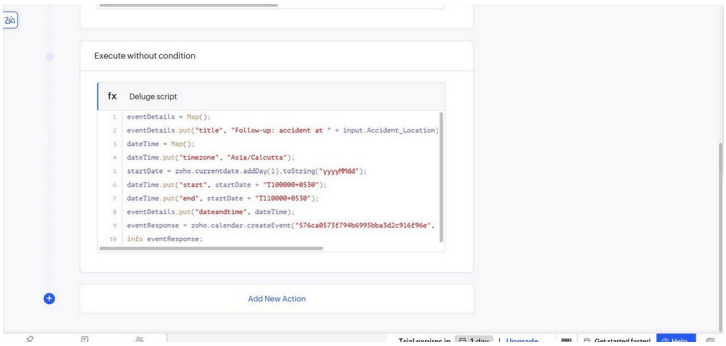


Figure 6.1: First Version of Script DS-03 Saved as an Action of the Workflow (before the BUG-06 fix)

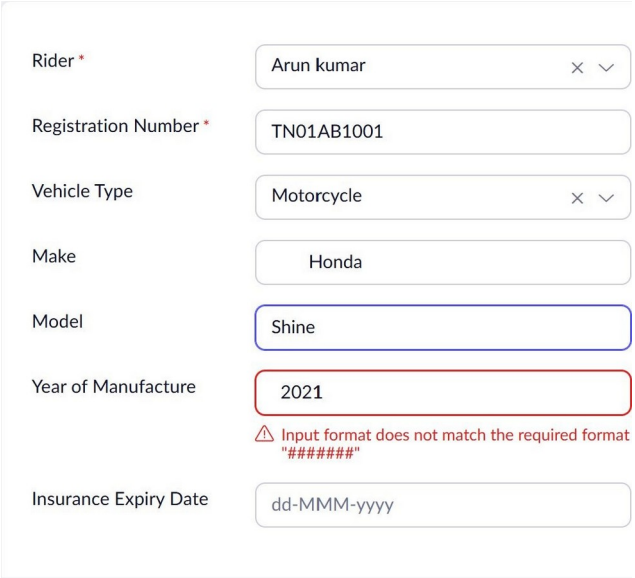


Figure 6.2: Form Validation Message on Year of Manufacture (BUG-01)

6.4 Defect Log

Table 6.4: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Vehicle form: “Input format does not match the required format ‘#######’” when entering Year of Manufacture 2021 (Fig. 6.2).	Number field format/digits setting did not accept a 4-digit year.	Field format changed in the builder; later vehicles saved with years 2020–2022.	Closed
BUG-02	Action Taken form: Accident Report lookup lists only numbers (1, 2, 3 …).	The only identifying field of Accident Report is the auto-number Report Number.	Accepted: the report number identifies the report. A richer display name is proposed.	Accepte d
BUG-03	Open Accident Cases showed 5 of 5 records, including a closed one (Fig. 5.10 before the filter).	The filter “Status not equal to Closed” had not been applied.	Filter added in the report settings; the report shows 4 of 4.	Closed
BUG-04	Dashboard pie chart showed one slice per record (5 slices).	Value type was “Actual data” on Accident Date and Time.	Changed to Aggregate count of Report Number by Severity.	Closed

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-05	Dashboard showed a Rider Report button and table instead of Open Accident Cases.	Wrong report chosen in the Report element.	Report element reconfigured to Open Accident Cases.	Closed
BUG-06	No Zoho Calendar event appears after submitting a report.	The built-in createEvent call returned LESS_THAN_MIN_ OCCURANCE; the calendar identifier and connection were correct. The reason for the error was not determined.	createEvent replaced by an invokeurl POST to the Calendar API; the event is now created (Fig. 6.5).	Closed
BUG-07	Sample reports 4 and 5 have accident dates in November 2026, later than the date of entry.	No validation on the Accident Date and Time field.	Left as test data; a validation is proposed.	Open

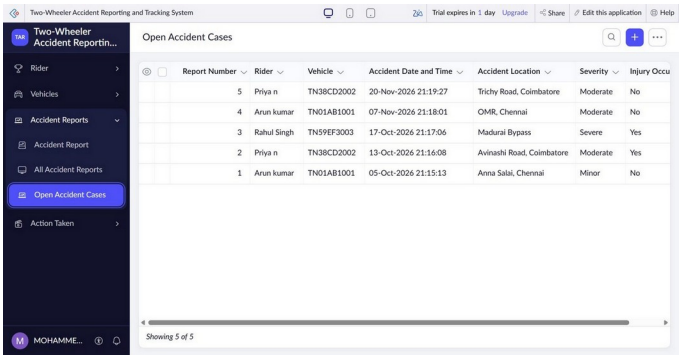


Figure 6.3: Open Accident Cases before the Filter Was Applied (5 of 5, BUG-03)

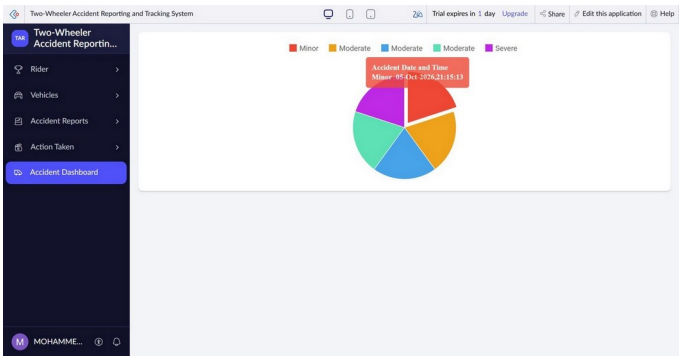


Figure 6.4: Dashboard Pie Chart Using Actual Data (BUG-04)

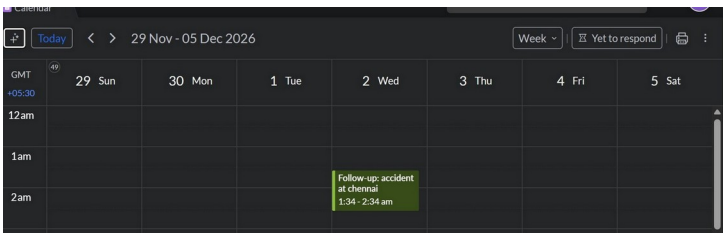
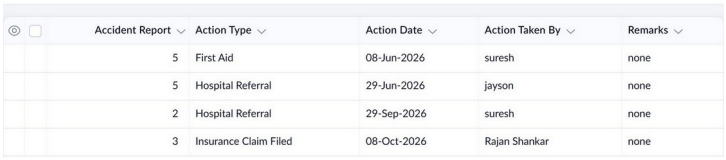


Figure 6.5: Zoho Calendar Showing the Follow-up Event Created for Report 6 (BUG-06 Fixed)



Accident Report	Action Type	Action Date	Action Taken By	Remarks
5	First Aid	08-Jun-2026	suresh	none
5	Hospital Referral	29-Jun-2026	jayson	none
2	Hospital Referral	29-Sep-2026	suresh	none
3	Insurance Claim Filed	08-Oct-2026	Rajan Shankar	none

Figure 6.6: Action Taken Report Listing Four Records

6.5 User Acceptance Testing

No feedback from other users has been recorded for this application yet. Table 6.5 is left for the student to complete after at least two people (for example a friend acting as a Rider and another as a Safety Officer) have tried the application.

Table 6.5: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken

CHAPTER 7

DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator provides a multi-stage deployment space (Deploy → Environments) with Stage and Production environments, and development takes place in the builder. After the trial ended, the application was backed up and restored in a new workspace, and on 31 October 2026 it was added to the environments and published. It was first published as version 1.0. Later the same day a minor version 1.1 with the calendar fix (BUG-06) was published to Stage and then to Production. The publish dialog asks for a title, a description and a Major or Minor type, shows the version change, and warns that the application is in maintenance mode until it is published. The Environments page now lists version 1.1 in Stage and version 1.1 in Production, both dated Oct 31, 2026, and the Development column shows “No changes available” (Figure 7.1). The publish menu offers “Publish to Stage” and “Publish to Production” (Figure 7.2, captured before the backup and restore).

The rollback plan is to keep the previous version in the Environments page and to republish it if a later change breaks the application; no rollback was exercised.



Figure 7.1: Environments Page Showing Version 1.1 in Stage and Production (31 Oct 2026)

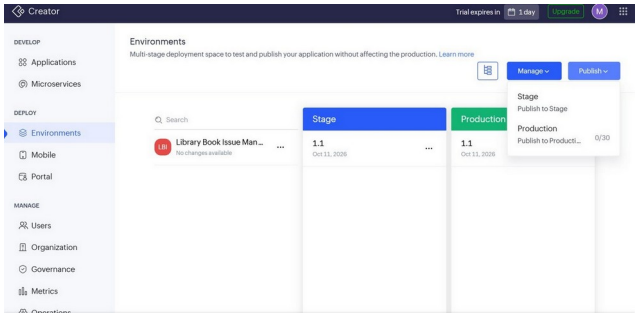


Figure 7.2: Publish Menu: Publish to Stage / Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (√)
1	All test cases executed in Development / Stage. (TC-02, TC-04, TC-06, TC-15 and TC-16 not run; TC-13 passed after the BUG-06 fix.)	✗

S.No.	Checklist Item	Done (√)
2	Sample/test data removed or isolated. (Seven sample reports remain.)	✗
3	Roles, permissions and users configured in Production. (Roles, permission sets and sharing rule defined; two test users assigned and signed in; not every role tested.)	Partial
4	Connections authorised in Production. (Connection authorization is the same for all environments.)	√
5	Schedules and workflows enabled. (Both form workflows show Enabled; no schedules exist.)	√
6	Application published and URL verified. (Stage 1.1 and Production 1.1 listed on 31 Oct 2026; the application URL was opened by signing in as the two test users.)	√
7	Version / release notes recorded. (Version 1.0 published after the restore; minor version 1.1 published with the calendar fix, each with a title and description in the publish dialog.)	√

7.2 Performance Optimisation

No performance measurements were taken, so no improvement is claimed. The design choices that help performance are: the Open Accident Cases report applies a filter so that closed cases are not loaded; the dashboard charts use an aggregate count instead of plotting every record (BUG-04); the scripts use no fetch statements or loops; and each report shows a limited set of columns. Table 7.2 records this honestly.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Filter “Status not equal to Closed”	Open Accident Cases	5 records listed	4 records listed	Fewer rows; load time not measured
Aggregate count instead of actual data	Severity pie chart	5 slices (one per record)	3 slices (one per severity)	Readable chart; load time not measured

Optimisation Applied	Component	Before	After	Improvement
No fetch or loop in scripts	DS-01, DS-02, DS-03	—	—	Not applicable

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Permission sets	Safety Officer and Rider sets: Access, View, Edit chosen per form, report and page; Rider has no edit or delete	Unauthorised changes to records
Role hierarchy	CEO → Safety Officer → Rider (Settings → Roles)	Uncontrolled visibility of records between levels
Data sharing rule (Read Only)	Accident Report: Rider role shared to Safety Officer role	Accidental editing through shared access
Personal-data flag	Emergency Contact Phone: “Contains personal data (PII)” ticked	Marking sensitive data for privacy handling
Mandatory and no-duplicate validation	Rider name, Phone number, Registration Number, Licence Number and others	Incomplete or duplicate records
Connection-level access	zoho_calendar_connection: access turned on per application	Use of the connection by unrelated applications

Not applied: field-level encryption (“Encrypt data” is not selected), IP or domain restrictions, two-factor authentication and record-level criteria. Authentication is by the account sign-in of the platform; the Deluge scripts contain no passwords or API keys.

7.4 Governance and Administration

Administration is done in Settings (Users, Permissions, Roles, Data Sharing) and in the Environments page. Naming follows a simple convention: one form per entity, reports named after their purpose (Open Accident Cases, Accidents by Severity) and workflows named after what they do. A dedicated audit trail, change history and backup/export configuration was not evidenced, and the application has no named maintainer other than the administrator who owns it. Figure 7.3 shows the administration page of the application.

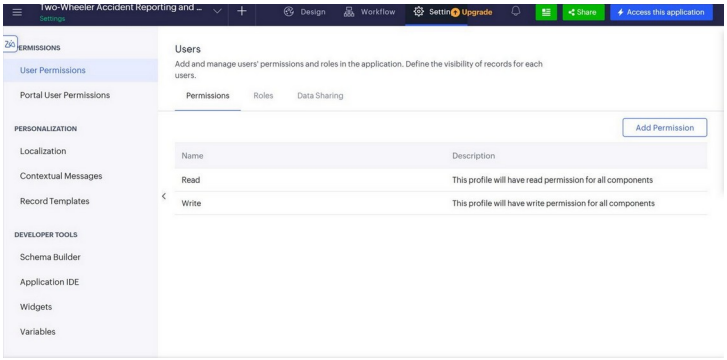


Figure 7.3: Administration: Settings → Users → Permissions

CHAPTER 8

RESULTS AND DISCUSSION

8.1 Application Output

The working application is presented through one realistic scenario, accident report 7. A rider's vehicle is involved in a severe accident on Salem Highway. In step 1 the report is entered in the Accident Report form with the Status left at its default value Reported (Figure 8.1). In step 2 the administrator receives the e-mail with the subject "New Accident Report Submitted – Severe" (Figure 8.2). In step 3 the stored record shows that the status was changed to Under Review by DS-02 (Figure 8.3). In step 4 the dashboard shows the charts and the open cases (Figure 8.4).

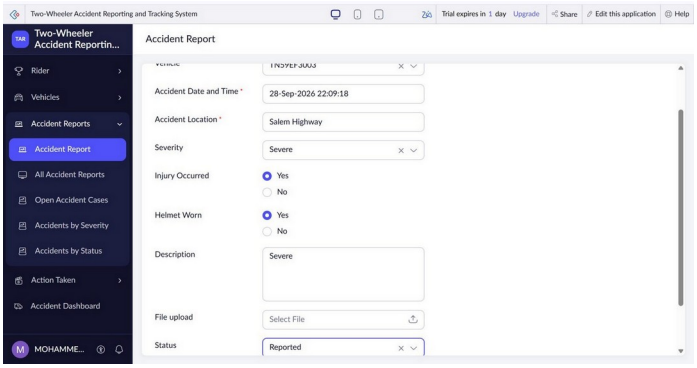


Figure 8.1: Scenario Step 1 – Severe Accident Report Entered with Status Reported

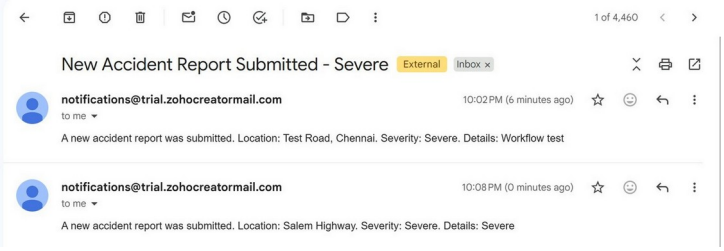


Figure 8.2: Scenario Step 2 – Notification E-mails Received (Test Road, Chennai and Salem Highway)

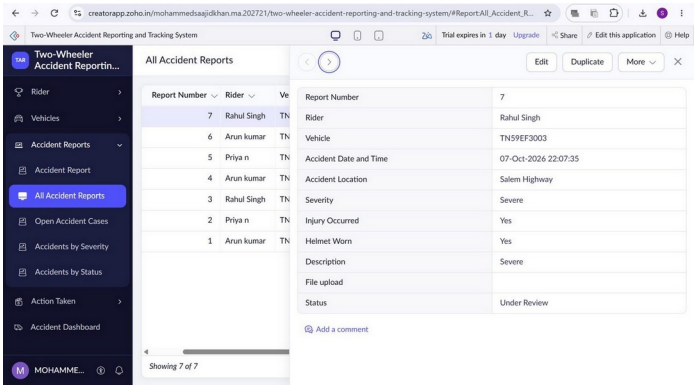


Figure 8.3: Scenario Step 3 – Stored Report 7 with Status Under Review

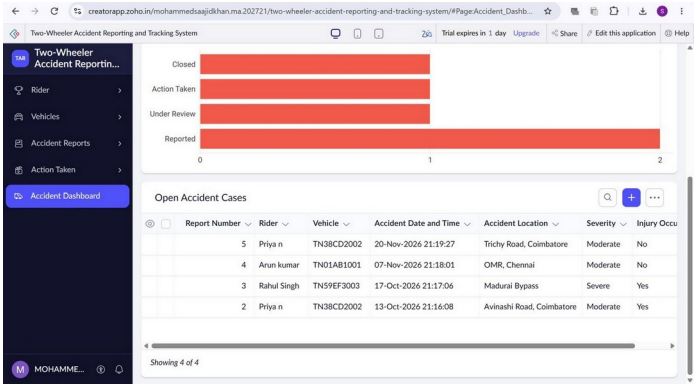


Figure 8.4: Scenario Step 4 – Dashboard with Open Accident Cases

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
1. Data model of four linked forms with validations	Sections 4.2, 4.3, 5.2	Achieved. Four forms, lookups, mandatory and no-duplicate validations (Fig. 5.3–5.8).
2. Automation with Deluge and workflows	Sections 5.5, 5.6	Achieved for e-mail notification (DS-01) and automatic status (DS-02); verified in Fig. 8.2 and 8.3.
3. Reports and dashboard	Sections 5.3, 5.4	Achieved. Open Accident Cases, two pivot charts and the Accident Dashboard (Fig. 5.10–5.16).
4. Integration with Zoho Calendar	Section 5.7	Achieved. Connection and script configured; the follow-up event is created at the accident date and time (BUG-06 fixed, Fig. 6.5).
5. Access control and deployment	Sections 5.8, 7.1	Largely achieved. Permission sets, hierarchy, sharing rule and Stage/Production publishing (version 1.1) done; two test users signed in and the sharing rule was observed (Section 5.8); every role was not tested systematically.

8.3 Comparison with the Existing System

No timing or error-rate measurements were made for either the informal process or the application, so Table 8.2 compares them qualitatively and no percentage improvements are claimed.

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Recording an accident	Notebook, call or phone gallery; structure varies	One form with fixed fields, lookups and drop-downs	Consistent, complete records
Handling of severe accidents	Depends on someone passing the news on	Automatic status Under Review and an e-mail with the severity	Severe cases are flagged without manual steps
Finding open cases	Read through the notes	Open Accident Cases report	Immediate list of cases that are not closed
Summary reporting	Counting by hand	Pivot charts and dashboard	Counts by severity and status on one page
Data accessibility and control	Whoever holds the notes sees everything	Permission sets, roles and sharing rule (configured; tried informally with two test users)	Controlled access once users are assigned

Table 8.3 records the quantities that can be read from the dashboard and report screenshots (Figures 5.10, 5.15 and 5.16). They describe the sample data at the time of the screenshots and are observations, not performance measurements.

Table 8.3: Observed Application State from the Screenshots

Measure	Observed Value	Source
Accident reports in the application	5 (report numbers 1 to 5)	Figure 5.15, Figure 5.16
Reports with status Reported	2	Figure 5.16
Reports with status Under Review	1	Figure 5.16
Reports with status Action Taken	1	Figure 5.16
Reports with status Closed	1	Figure 5.16
Open Accident Cases shown (Closed hidden)	4 of 4	Figure 5.10, Figure 5.16

8.4 Limitations of the Low-Code Approach

The following constraints and issues were met while building this application.

- **Integration debugging.** The Zoho Calendar action could be configured in builders, but when no event appeared the platform showed no visible error. The cause could only be seen after the response of the call was e-mailed to the administrator: the built-in createEvent call kept returning LESS_THAN_MIN_OCCURANCE, and a direct invokeurl call to the Calendar API worked (BUG-06).
- **User invitations.** After the application was restored into a new workspace, inviting some addresses failed (the Accept Invitation button did nothing and adding the address again gave a Zoho error). Two other addresses worked and were used as test users.
- **Editions and trial limits.** The application was built on a trial edition that expires soon, and the Settings tab shows an “Upgrade” badge, so user invitation and some governance features could not be assumed to be available.
- **Lookup display.** A lookup shows only the field chosen as display value, so Accident Report appears as a bare number in the Action Taken form (BUG-02). A formula or concatenated display field would be needed for a friendlier label.
- **Validation rules.** Standard field settings do not stop future accident dates; this would need a Deluge validation script.
- **Platform dependency.** The application exists only inside Zoho Creator, and the logic is written in Deluge, which is specific to Zoho (vendor lock-in).
- **Scripting scope.** Deluge covered every need of this project, but its statement and API-call limits were not tested because the scripts are small.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of the application is **no-code configuration**: the four forms, their lookups and validations, the reports, the charts, the dashboard page, the roles, permission sets and sharing rule, and the Stage/Production publishing. **Low-code** was needed for three small Deluge scripts: the notification, the automatic status rule and the calendar event. **Pro-code** was not needed. A move to pro-code would be justified if the system had to accept reports from a mobile app or roadside device through the Creator REST API, to analyse accident locations on a map with heavier processing in a Catalyst function, or to integrate with an insurer’s external system, because these go beyond what forms, reports and short scripts can do comfortably.

CHAPTER 9

CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

The Two-Wheeler Accident Reporting and Tracking System shows how an informal accident-recording practice can be replaced by a small low-code application. Four linked forms store riders, vehicles, accident reports and actions taken. Two form workflows e-mail the administrator and move Severe and Fatal reports to Under Review automatically. A filtered list, two pivot charts and a dashboard give an immediate picture of the open cases, and permission sets, a role hierarchy and a sharing rule control access. The application was published to the Stage and Production environments (version 1.1).

The work also has limits that are recorded openly: role-based access was tried only informally with two test users and not tested for every role, no timing measurements were made, and feedback from other users has not yet been collected. These items are the first to address before the application is used for real accident records.

9.2 Future Enhancements

- Find out why the built-in createEvent call returned an error, and make the calendar step check the response and report a failure instead of failing silently.
- Add record-level criteria so that a rider sees only own reports, and test every role systematically, including that a Rider cannot edit reports (TC-15).
- Add a scheduled workflow that reminds the Safety Officer of cases that stay in Under Review for too long (proposed FR-14).
- Add an approval process or Blueprint to enforce the order of statuses before a case is closed.
- Notify the emergency contact of the rider for Severe and Fatal accidents (proposed FR-13), and add a validation that rejects future accident dates.
- Publish a customer portal or use the Creator mobile app so that riders can report accidents from the roadside, with the location filled in automatically.
- Show accident locations on a Map report and analyse hot-spots; encrypt sensitive fields.

9.3 Learning Outcomes and Career Relevance

Building this application gave practice in the main parts of Zoho Creator: data modelling with forms and lookups, validations, list and pivot-chart reports, UI Builder pages, form

workflows, Deluge scripting, connections, permission sets and roles, and deployment through environments. It also showed how testing and debugging work in a low-code platform, including the cases where a fix is not found immediately. These skills relate to the roles of citizen developer, low-code developer, solutions consultant and automation engineer, and they provide a basis for moving to pro-code extensions when requirements outgrow the platform.

REFERENCES

[1] Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, “Characteristics and Challenges of Low-Code Development: The Practitioners’ Perspective,” in *Proc. 15th ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, 2021. [Online]. Available: https://arxiv.org/abs/2107.07482 (Accessed: 22 Oct. 2026).

[2] M. A. Al Alamin, S. Malakar, G. Uddin, S. Afroz, T. B. Haider and A. Iqbal, “An Empirical Study of Developer Discussions on Low-Code Software Development Challenges,” arXiv:2103.11429, 2021. [Online]. Available: https://arxiv.org/abs/2103.11429 (Accessed: 22 Oct. 2026).

[3] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 22 Oct. 2026).

[4] Zoho Corporation, “Deluge Script Reference.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 22 Oct. 2026).

[5] Zoho Corporation, “Create event in Zoho Calendar,” Zoho Deluge Help. [Online]. Available: https://www.zoho.com/deluge/help/calendar/create-event.html (Accessed: 1 Nov. 2026).

Note: only sources actually cited in the text are listed. No other statistics, quotations or literature are used in this report.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Y = implemented, Partial = implemented only in part (see note), N = not implemented. The checklist reflects the application as built and tested and should be re-checked against the live application before the demonstration.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Menu: Rider, Vehicles, Accident Reports, Action Taken, Accident Dashboard	5.1	Fig. 5.1, 5.2
II	Forms with ≥ 8 distinct field types	Y	Name, Phone, Email, Single Line, Multi Line, Number, Date, Date-Time, Drop Down, Radio, Lookup, Auto Number, File Upload (13)	5.2	Fig. 5.3–5.6
II	Lookup field (relationship between forms)	Y	Vehicle, Accident Report, Action Taken	4.2, 5.2	Fig. 4.2, 5.4
II	Subform (parent–child data)	N	Not used	4.2	—
II	Field validations / layout rules	Y	Mandatory and no-duplicate validations; no layout rules	5.2	Fig. 5.3, 5.4
II	List report with filter, grouping, custom action	Partial	Open Accident Cases has a filter; no grouping or custom action	5.3	Fig. 5.9, 5.10
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	—
II	Pivot table / chart report	Y	Accidents by Severity, Accidents by Status	5.3	Fig. 5.12, 5.13
II	Page / dashboard built with UI Builder	Y	Accident Dashboard	5.4	Fig. 5.14–5.16
II	Deluge – form events (validate / success / field actions)	Y	Validations on form submission; Successful form submission	5.5	Code 5.1, 5.2
II	Deluge – functions, fetch, loops, collections	Partial	Map collection used in DS-03; no function, fetch or loop	5.5	Code 5.3
II	Email / in-app notifications	Y	sendmail in DS-01	5.5, 5.6	Fig. 8.2
II	Form workflow	Y	Two form workflows	5.6	Fig. 5.20, 5.21
II	Scheduled workflow	N	Not used	5.6	—
II	Approval process	N	Not used	5.6	—
II	Blueprint	N	Not used	5.6	—
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Y	Test cases, defect log, debugging figures	6	Fig. 6.1–6.5

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Integration with a Zoho service	Y	Zoho Calendar connection; follow-up event created at the accident time	5.7	Fig. 5.22–5.25, 6.5
III	Connection / REST API call (invokeurl)	Y	invokeurl POST to the Zoho Calendar API through the connection	5.7	Code 5.3
III	Webhook or Creator REST API	N	Not used	5.7	—
III	Users, roles and permission profiles	Y	Roles hierarchy and two permission sets; two test users assigned and signed in	5.8	Fig. 5.26–5.32
III	Record-level access criteria / customer portal	Partial	Data sharing rule only; no criteria, no portal	5.8	Fig. 5.30
III	Zia / AI feature	N	Not used in the application	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Pivot charts and dashboard charts; Zoho Analytics not used	5.9	Fig. 5.12–5.15
IV	Performance optimisation applied & measured	N	Design choices only; nothing measured	7.2	Table 7.2
IV	Deployment across environments	Y	Stage 1.1 and Production 1.1 (31 Oct 2026)	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Partial	Permissions, PII flag and connection used; no encryption	7.3	Table 7.3
IV	Governance (audit trail, administration)	N	Administration pages only; no audit trail evidenced	7.4	Fig. 7.3
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not used; justified in 5.10	5.10	—
V	Low-code limitations & spectrum analysis	Y	Discussed	8.4, 8.5	—
—	Mobile app / portal access	N	Not configured	5.11	—

Total features implemented: 16 fully, 4 partly, 12 not implemented, out of 32.

APPENDIX B – COURSE OUTCOME MAPPING

The self-assessed level (1 – Low, 2 – Medium, 3 – High) is left for the student; the examiner may revise it.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO 1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO 2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO 3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO 4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO 5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The scripts below are exactly as saved in the application. Comments were added for this report.

C.1 Workflow “Notify on New Accident Report” (Accident Report form)

```
// Script ID : DS-01
// Location  : Accident Report -> Created -> Successful form
submission -> Notify on New Accident Report -> action 1
// Purpose   : E-mail the administrator when a report is submitted
sendmail
[
from :zoho.adminuserid
to :zoho.adminuserid
subject :"New Accident Report Submitted - " + input.Severity
message :"A new accident report was submitted. Location: " +
input.Accident_Location + ". Severity: " + input.Severity + ".
Details: " + input.Description
]
```

C.2 Workflow “Auto Review for Severe Accidents” (Accident Report form)

```
// Script ID : DS-02
// Location  : Accident Report -> Created -> Validations on form
submission -> Auto Review for Severe Accidents
// Purpose   : Set the status of Severe and Fatal reports to Under
Review before saving
if(input.Severity == "Severe" || input.Severity == "Fatal")
{
input.Status = "Under Review";
}
```

C.3 Workflow “Notify on New Accident Report” – calendar action

```
// Script ID : DS-03
// Location  : Accident Report -> Created -> Successful form
submission -> Notify on New Accident Report -> action 2
// Purpose   : Create a one-hour follow-up event in Zoho Calendar
at the accident date and time
// Status    : Working after the BUG-06 fix (createEvent replaced
by invokeurl)
eventDetails = Map();
eventDetails.put("title", "Follow-up: accident at " +
input.Accident_Location);
startUTC = input.Accident_Date_and_Time.subMinutes(330);
endUTC = input.Accident_Date_and_Time.subMinutes(270);
dateTime = Map();
dateTime.put("timezone", "Asia/Calcutta");
dateTime.put("start", startUTC.toString("yyyyMMdd") + "T" +
startUTC.toString("HHmmss") + "Z");
dateTime.put("end", endUTC.toString("yyyyMMdd") + "T" +
endUTC.toString("HHmmss") + "Z");
eventDetails.put("dateandtime", dateTime);
params = Map();
params.put("eventdata", eventDetails.toString());
eventResponse = invokeurl
[
url:
"https://calendar.zoho.in/api/v1/calendars/576ca0573f794b6995bba3d2
c916f96e/events"
type: POST
parameters: params
connection: "zoho_calendar_connection"
];
info eventResponse;
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Sign in to Zoho Creator (India data centre, creator.zoho.in) with your Zoho account and open the application “Two-Wheeler Accident Reporting and Tracking System”. The menu on the left shows the sections Rider, Vehicles, Accident Reports, Action Taken and the page Accident Dashboard. Mobile access has not been configured. https://mohammedsaajidkhanma202344.zohocreatorportal.in

D.2 Instructions for the Administrator

1. Open **Rider → Rider** and enter the rider’s name, phone number and, if available, e-mail, licence number, address and emergency contact; click Submit.
2. Open **Vehicles → Vehicle**, choose the Rider, enter the registration number (it must be unique), the vehicle type and other details; click Submit.
3. To see all riders or vehicles, open **Rider Report** or **All Vehicles**.
4. Open **Accident Dashboard** to see the charts by severity and status and the list of open cases.
5. The notification e-mail for every new accident report arrives in the administrator’s mailbox.
6. Manage roles, permission sets and the data sharing rule in **Settings → Users**.

D.3 Instructions for the Safety Officer

1. Open **Accident Reports → Open Accident Cases** to see every case that is not closed.
2. Click a record to see its details, then click **Edit**, change **Status** to Under Review, Action Taken or Closed, and save.
3. Open **Action Taken → Action Taken**, choose the report number, the action type, the date and the person who acted, and submit.
4. Use **Accident Dashboard** for the overall picture.

D.4 Instructions for the Rider

1. Open **Accident Reports → Accident Report**.
2. Choose your name in **Rider** and your vehicle in **Vehicle**; enter the accident date and time and the location (mandatory).
3. Choose the severity, say whether anyone was injured and whether a helmet was worn, describe what happened and attach a photo if you have one.
4. Click **Submit**. The report is numbered automatically; Severe or Fatal reports are placed under review straight away.

The Safety Officer and Rider instructions describe the roles as designed. Two test users were assigned to the Rider and Safety Officer roles and signed in, but every role was not tested systematically (Section 5.8).